DOCUMENTO DE ANÁLISIS, DISEÑO Y OPERACIÓN

IT Incident Hub

Plataforma web de gestión de incidentes de TI con ciclo de entrega continuo DevOps

Versión del sistema	1.1.0
Versión del documento	2.0
Fecha	26 de octubre de 2026
Estado	Versión final para entrega
Contexto	Proyecto integrador DevOps (Módulos 1 a 10)
Repositorio	Azure Repos (principal) y GitHub (espejo con GitHub Actions)

Equipo (simulado de 3 integrantes, cuentas independientes): Jesús Soto (SotoJ08) · DharaGar29 · Edd210

Control del documento

Historial de versiones

Tabla 1. Versiones del documento

Versión	Fecha	Descripción	Responsable
1.0	2026-10-24	Documentación técnica inicial en el repositorio (`docs/*.md`) para la versión 1.0.0 del sistema.	Equipo
1.1	2026-10-25	Actualización por la versión 1.1.0: inicio de sesión, tres roles y asignación automática por categoría.	Equipo
2.0	2026-10-26	Documento de análisis, diseño y operación consolidado (este documento): requerimientos, casos de uso, arquitectura, datos, API, seguridad, DevOps, pruebas, manuales y trazabilidad.	Equipo

Roles del equipo y responsabilidades

Tabla 2. Equipo del proyecto

Integrante (cuenta)	Rol en el proyecto	Responsabilidades principales
Jesús Soto (SotoJ08)	Líder técnico · Backend y DevOps	API REST, modelo de datos, pipelines, ambientes, release y documentación
DharaGar29	Frontend	Interfaz React: login, lista, detalle, formulario, historial y dashboard
Edd210	Pruebas y seguridad	Pruebas automáticas, escáner de secretos, análisis de dependencias y permisos por rol

> **Transparencia sobre el equipo**
> Es un proyecto individual; el equipo de tres integrantes es **simulado** mediante tres cuentas independientes con correo propio. El historial de Git conserva la autoría real de cada commit (la distribución es desigual y los primeros commits se hicieron con una sola cuenta; no se reescribió el historial). Ver el apartado 16.

Documentos relacionados

- `README.md` y `CHANGELOG.md`: guía de uso del repositorio y historial de versiones.
- `docs/entrega.md`: lista de verificación contra el enunciado del proyecto (secciones 1 a 9).
- `docs/evidencia/`: evidencias exportadas (backlog, Pull Requests, corridas de pipeline, GitHub Actions, historial Git).
- `docs/architecture.md`, `security.md`, `release-strategy.md`, `dependencies.md`, `technical-debt.md`, `branching.md`, `ci-failure-diagnosis.md`: documentación técnica detallada por tema, que este documento consolida.

Tabla de contenido

Control del documento 2
- Historial de versiones 2
- Roles del equipo y responsabilidades 2
- Documentos relacionados 2

1. Introducción 5
- 1.1 Propósito 5
- 1.2 Alcance 5
- 1.3 Audiencia 5
- 1.4 Definiciones y abreviaturas 6
- 1.5 Referencias 6
- 1.6 Convenciones del documento 6

2. Descripción general del sistema 7
- 2.1 Contexto y problema 7
- 2.2 Objetivos 7
- 2.3 Perspectiva del producto 7
- 2.4 Actores y roles 9
- 2.5 Supuestos, dependencias y restricciones 9
- 2.6 Stack tecnológico 9

3. Especificación de requerimientos 11
- 3.1 Requerimientos funcionales 11
- 3.2 Requerimientos no funcionales 12
- 3.3 Reglas de negocio 12

4. Modelo de casos de uso 14
- 4.1 Diagrama de casos de uso 14
- 4.2 Catálogo de casos de uso 15
- 4.3 Especificación de los casos de uso principales 15

5. Arquitectura del sistema 18
- 5.1 Visión general y estilo arquitectónico 18
- 5.2 Backend 18
- 5.3 Frontend 18
- 5.4 Decisiones de diseño 19
- 5.5 Vista de despliegue 19

6. Diseño de datos 21
- 6.1 Modelo entidad-relación 21
- 6.2 Diccionario de datos 22
- 6.3 Índices y datos de ejemplo 23

7. Diseño del comportamiento 25
- 7.1 Ciclo de vida del incidente 25
- 7.2 Secuencia: iniciar sesión y registrar un incidente 27
- 7.3 Algoritmo de asignación automática 28
- 7.4 Cálculo del alcance de visibilidad 28

8. Especificación de la API REST 29
- 8.1 Convenciones 29
- 8.2 Catálogo de endpoints 29
- 8.3 Ejemplos 30

9. Seguridad 31
- 9.1 Principios 31
- 9.2 Autenticación 31
- 9.3 Autorización: matriz de permisos 31
- 9.4 Modelo de amenazas resumido 32

9.5 Gestión de secretos ... 32
9.6 Controles automáticos ... 33

10. DevOps y gestión del ciclo de vida ... 34
10.1 Trazabilidad de extremo a extremo ... 34
10.2 Gestión del trabajo en Azure Boards ... 36
10.3 Estrategia de ramas y Pull Requests ... 37
10.4 Integración continua ... 38
10.5 Fallos controlados del pipeline ... 39
10.6 Entrega por ambientes y release ... 39
10.7 Versionado semántico ... 40
10.8 Análisis de dependencias ... 41

11. Aseguramiento de la calidad ... 42
11.1 Estrategia de pruebas ... 42
11.2 Plan de pruebas de aceptación ... 42
11.3 Resultados y métricas ... 43

12. Manual de instalación y operación ... 44
12.1 Requisitos ... 44
12.2 Instalación con PostgreSQL (Docker) ... 44
12.3 Modo demostración sin Docker ... 44
12.4 Variables de entorno ... 45
12.5 Ejecución de pruebas y verificaciones ... 45
12.6 Operación ... 45
12.7 Solución de problemas ... 45

13. Manual de usuario ... 47
13.1 Acceso al sistema ... 47
13.2 Solicitante: reportar y seguir un incidente ... 47
13.3 Técnico: atender incidentes ... 48
13.4 Administrador: asignar y supervisar ... 49
13.5 Mensajes de la interfaz ... 50

14. Matriz de trazabilidad ... 51

15. Riesgos, deuda técnica y limitaciones ... 52
15.1 Registro de deuda técnica ... 52
15.2 Riesgos del proyecto ... 52
15.3 Limitaciones conocidas ... 53
15.4 Trabajo futuro ... 53

16. Gestión del proyecto ... 54
16.1 Cronograma de hitos ... 54
16.2 Contribuciones por integrante ... 54
16.3 Evidencias disponibles ... 54
16.4 Conclusiones ... 55

1. Introducción

1.1 Propósito

Este documento describe el análisis, el diseño, la construcción, la operación y el proceso de entrega de **IT Incident Hub**, una plataforma web para registrar, asignar y dar seguimiento a incidentes de tecnología de la información (TI). Está dirigido a quien deba evaluar, mantener, desplegar o ampliar el sistema, y reúne en un solo lugar lo que de otro modo estaría disperso en el repositorio.

Sigue la estructura habitual de una especificación de software (basada en las recomendaciones de ISO/IEC/IEEE 29148 para requerimientos y en el modelo de vistas de arc42 y C4 para arquitectura), adaptada a un proyecto académico de tamaño pequeño.

1.2 Alcance

El sistema cubre el ciclo completo de un incidente de TI: registro por un solicitante, asignación a un técnico (automática o manual), atención con cambios de estado y comentarios, cierre, consulta con filtros e indicadores. Incluye autenticación con tres roles y control de acceso aplicado en el servidor.

El alcance de proceso incluye la gestión del trabajo en Azure Boards, el control de versiones con ramas y Pull Requests, la integración continua en Azure Pipelines (agente autohospedado) y GitHub Actions, la gestión de secretos, el análisis de dependencias, el versionado semántico y la entrega por ambientes (dev, staging y producción) con aprobaciones.

Tabla 3. Fuera de alcance

Elemento	Razón
Notificaciones por correo o mensajería	No requerido por el enunciado; se puede añadir sobre la bitácora de eventos
Adjuntos y registro de tiempos (SLA)	Fuera del modelo de datos actual
Gestión de usuarios desde la interfaz	Los usuarios se cargan por migración y semilla; no hay pantalla de administración de usuarios
Despliegue en un servidor externo o nube	El despliegue por ambientes es local sobre el agente autohospedado (ver apartado 10.6)
Paginación del listado	Registrada como deuda técnica D7

1.3 Audiencia

Tabla 4. Lectores previstos y qué buscar

Lector	Capítulos recomendados
Evaluador o docente	1, 2, 3, 10, 11, 15 y 16; luego `docs/entrega.md`
Analista funcional	2, 3, 4, 7 y 13
Desarrollador	5, 6, 7, 8, 9 y 11
Operador o DevOps	9, 10 y 12

Lector	Capítulos recomendados
Usuario final	13

1.4 Definiciones y abreviaturas

Tabla 5. Glosario

Término	Definición
Incidente	Evento que interrumpe o degrada un servicio de TI y que un usuario reporta para su atención
Solicitante	Usuario que reporta incidentes y consulta los suyos
Técnico	Usuario que atiende incidentes de las categorías de su especialidad
Administrador	Usuario con visibilidad y control total; asigna técnicos y gestiona estados
Bitácora	Registro cronológico de comentarios, cambios de estado y asignaciones de un incidente
API REST	Interfaz HTTP con recursos y verbos estándar que usa la interfaz web
SPA	Aplicación de una sola página (Single Page Application)
JWT / token HS256	Token firmado con HMAC-SHA256 que identifica la sesión de un usuario
scrypt	Función de derivación de claves resistente a ataques de fuerza bruta con hardware especializado
CI/CD	Integración continua y entrega continua
Work Item / AB#ID	Elemento de trabajo de Azure Boards; el prefijo `AB#` en commits y PR lo enlaza
PR	Pull Request: solicitud de fusión de una rama a `main` con revisión
SemVer	Versionado semántico MAJOR.MINOR.PATCH
PAT	Personal Access Token de Azure DevOps
Deuda técnica	Decisión o carencia de implementación que facilita avanzar hoy pero encarece el trabajo futuro

1.5 Referencias

- Enunciado del proyecto integrador DevOps (documento `Proyecto_Modulos_1_10.pdf`).
- ISO/IEC/IEEE 29148:2018 · Ingeniería de requerimientos.
- arc42 (plantilla de documentación de arquitectura) y modelo C4 de Simon Brown.
- Versionado Semántico 2.0.0 (semver.org) y Keep a Changelog 1.1.0.
- OWASP ASVS y OWASP Top 10 como referencia de controles de seguridad.

1.6 Convenciones del documento

Los identificadores siguen el esquema **RF-nn** (requerimiento funcional), **RNF-nn** (no funcional), **RN-nn** (regla de negocio), **CU-nn** (caso de uso), **CP-nn** (caso de prueba) y **D-n** (deuda técnica). El nombre de archivos, rutas y comandos aparece en `tipografía de código`. Las tablas y figuras están numeradas; las limitaciones conocidas se señalan de forma explícita en recuadros.

2. Descripción general del sistema

2.1 Contexto y problema

Las áreas de soporte de TI de una institución reciben fallas de equipos, sistemas, accesos y comunicaciones por canales dispersos (llamadas, mensajes, correos). Sin un registro único es difícil saber quién atiende qué, en qué estado está cada caso y cuántos incidentes críticos siguen abiertos. El resultado típico es tiempo perdido, casos olvidados y falta de métricas para decidir.

IT Incident Hub centraliza el registro y el seguimiento: cada incidente tiene un responsable, un estado dentro de un flujo definido y una bitácora auditable, y un panel resume la carga de trabajo.

2.2 Objetivos

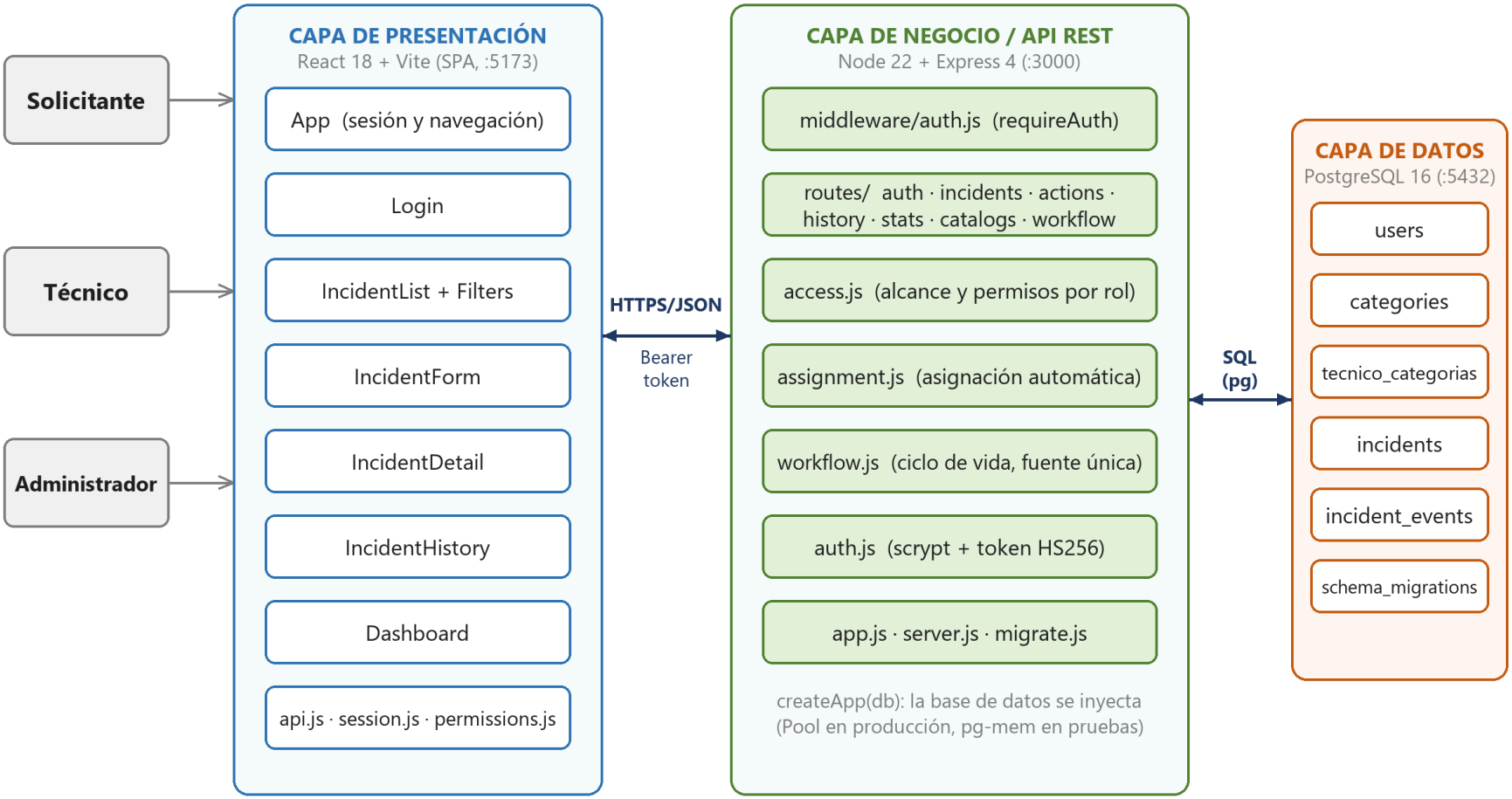
Tabla 6. Objetivos del sistema

Código	Objetivo	Indicador de cumplimiento
OBJ-1	Registrar incidentes con la información mínima necesaria	Formulario con validación; RF-03
OBJ-2	Reducir el tiempo de asignación	Asignación automática al crear (RF-04), sin intervención humana cuando la categoría tiene técnico
OBJ-3	Dar visibilidad adecuada a cada perfil	Alcance por rol aplicado en el servidor (RN-05)
OBJ-4	Garantizar trazabilidad de cada acción	Bitácora automática (RF-11)
OBJ-5	Ofrecer métricas de la operación	Dashboard (RF-12)
OBJ-6	Demostrar un ciclo de entrega DevOps verificable	Trazabilidad de Work Item a release (capítulo 10)

2.3 Perspectiva del producto

El sistema es una aplicación web de tres capas independiente: una SPA en React que consume una API REST en Node.js, la cual persiste en PostgreSQL. No depende de otros sistemas institucionales (no integra con directorio activo ni correo en esta versión). La Figura 1 muestra la arquitectura lógica; el capítulo 5 la detalla.

Arquitectura lógica en capas de IT Incident Hub



Todas las rutas, salvo /health, /api/auth/login y /api/workflow, exigen el encabezado Authorization: Bearer <token>.

Figura 1. Arquitectura lógica en capas de IT Incident Hub

2.4 Actores y roles

Tabla 7. Actores del sistema

Actor	Descripción	Alcance de visibilidad	Capacidades
Solicitante	Personal que reporta fallas	Solo los incidentes que reportó	Crear incidentes, consultar los suyos, comentar, ver su dashboard
Técnico	Personal de soporte con una o más especialidades (categorías)	Incidentes de las categorías que atiende	Crear incidentes, cambiar el estado de los asignados a él, comentar, ver el dashboard de sus categorías
Administrador	Responsable del servicio de TI	Todos los incidentes	Todo lo anterior sin restricción de alcance, más asignar y reasignar técnicos
Sistema	Actor interno	No aplica	Asignar automáticamente al crear; registrar la bitácora

2.5 Supuestos, dependencias y restricciones

Tabla 8. Supuestos y restricciones

Tipo	Descripción
Supuesto	Cada categoría de incidente puede tener cero o más técnicos de especialidad; la categoría «Otros» no tiene técnico y requiere asignación manual
Supuesto	Los usuarios y sus especialidades se cargan por migración y semilla (no existe aún su administración en pantalla)
Restricción técnica	Node.js 22 (o 20.19+), PostgreSQL 16 y Docker Desktop para la base de datos local
Restricción de proceso	Un único repositorio, backlog en Azure Boards, CI en Azure Pipelines con agente autohospedado y GitHub Actions
Restricción organizacional	Proyecto individual con equipo simulado; las aprobaciones de PR las da la misma identidad de Azure DevOps
Dependencia	Cuenta de Azure DevOps (organización `jsoto1`) y cuenta de GitHub (`SotoJ08`)

2.6 Stack tecnológico

Tabla 9. Tecnologías utilizadas

Capa	Tecnología	Versión	Justificación
Frontend	React + Vite	React 18 · Vite 8	Interfaz reactiva con herramientas de compilación rápidas
Backend	Node.js + Express	Node 22 · Express 4	API REST simple, un solo lenguaje en todo el stack
Datos	PostgreSQL	16	Base relacional con restricciones CHECK, claves foráneas e índices

Capa	Tecnología	Versión	Justificación
Seguridad	`crypto` de Node (scrypt, HMAC-SHA256)	Nativo	Sin dependencias adicionales ni superficie extra de vulnerabilidades
Pruebas backend	`node --test`, supertest, pg-mem	Nativo / 3 / 3	Pruebas de API sin servidor de base de datos real
Pruebas frontend	Vitest + Testing Library	Vitest 5	Pruebas de componentes y flujos de interfaz
CI/CD	Azure Pipelines y GitHub Actions	-	Pipeline principal con agente propio y flujo complementario
Contenedores	Docker Compose	-	Base de datos local reproducible

3. Especificación de requerimientos

3.1 Requerimientos funcionales

La columna **Prio.** usa la escala MoSCoW simplificada (A = alta/imprescindible, M = media, B = baja). Todos los requerimientos listados están implementados y verificados por pruebas automáticas (ver matriz de trazabilidad, capítulo 14).

Tabla 10. Requerimientos funcionales

ID	Descripción	Prio.	Caso de uso
RF-01	Iniciar sesión con correo y contraseña. El sistema entrega un token con vigencia de 8 horas.	A	CU-01
RF-02	Cerrar sesión y volver al login cuando el token es inválido, está alterado o expiró (respuesta 401).	A	CU-01
RF-03	Registrar un incidente con título (máx. 200), descripción, categoría y prioridad (por defecto BAJA). El solicitante es el usuario de la sesión.	A	CU-02
RF-04	Asignar automáticamente al crear: al técnico que atiende la categoría con menos incidentes abiertos; si no hay técnico, el incidente queda NUEVO.	A	CU-09
RF-05	Listar incidentes, los más recientes primero, limitados al alcance del rol.	A	CU-03
RF-06	Filtrar el listado por estado, prioridad, categoría y técnico, con combinación de filtros.	M	CU-03
RF-07	Ver el detalle de un incidente dentro del alcance del usuario.	A	CU-04
RF-08	Asignar o reasignar técnico (solo administrador). El técnico debe atender la categoría; solo en estados NUEVO, ASIGNADO y EN_PROCESO.	A	CU-07
RF-09	Cambiar el estado según el flujo (administrador o técnico asignado). No se avanza sin técnico asignado. Un incidente RESUELTO puede reabrirse.	A	CU-06
RF-10	Agregar comentarios (máx. 2000 caracteres) a incidentes visibles; no se admiten en incidentes CERRADO.	M	CU-05
RF-11	Registrar automáticamente en la bitácora comentarios, cambios de estado y asignaciones; consultar en orden cronológico con filtro por tipo.	A	CU-04
RF-12	Mostrar indicadores: total, abiertos, cerrados, críticos, sin asignar, y distribución por estado, categoría y técnico, limitados al rol.	M	CU-08
RF-13	Consultar catálogos de categorías y de usuarios por rol (los técnicos incluyen sus categorías), sin exponer correos ni hashes.	M	CU-02, CU-07
RF-14	Publicar las reglas del flujo de estados (`GET /api/workflow`) como fuente única para la interfaz.	B	CU-06

3.2 Requerimientos no funcionales

Tabla 11. Requerimientos no funcionales

ID	Categoría	Descripción y forma de verificación
RNF-01	Seguridad	Contraseñas almacenadas solo como hash scrypt con sal y comparación en tiempo constante; token HS256 con firma verificada; `AUTH_SECRET` obligatoria con `NODE_ENV=production`. Verificado por `auth.test.js` y `auth-routes.test.js`.
RNF-02	Seguridad	La autorización se aplica en el servidor en cada petición: 404 fuera de alcance, 403 sin permiso. El rol y las categorías se leen de la base en cada petición. Verificado por `permissions.test.js` y `access.test.js`.
RNF-03	Integridad de datos	Dominios controlados con restricciones CHECK, claves foráneas e índices. Verificado por `schema.test.js`.
RNF-04	Mantenibilidad	Reglas de negocio en módulos puros (`workflow.js`, `access.js`, `assignment.js`); la base de datos se inyecta en `createApp(db)`.
RNF-05	Calidad	202 pruebas automáticas (backend 108, frontend 67, utilidades DevOps 27) ejecutadas en cada corrida del pipeline.
RNF-06	Portabilidad	Ejecución en Windows y Linux con Node 22; base de datos local con Docker Compose; modo demostración sin Docker.
RNF-07	Usabilidad	Interfaz que muestra solo las acciones permitidas al rol, mensajes de error claros y gráficos accesibles (barras CSS con texto).
RNF-08	Trazabilidad	Cada cambio enlaza Work Item (AB#ID), rama, commit, Pull Request, corrida de pipeline y versión.
RNF-09	Gestión de secretos	Ningún secreto en el repositorio; escáner automático en cada corrida (`check-secrets.js`).
RNF-10	Entrega	Un artefacto único se promueve de dev a staging y producción con aprobación manual y rollback.
RNF-11	Dependencias	`npm audit` sin vulnerabilidades high/critical; el pipeline falla en caso contrario.
RNF-12	Rendimiento	No se definió un objetivo cuantitativo; el listado no pagina (deuda D7) y se considera adecuado para el volumen de un proyecto académico.

3.3 Reglas de negocio

Tabla 12. Reglas de negocio

ID	Regla
RN-01	El ciclo de vida es **NUEVO → ASIGNADO → EN_PROCESO → RESUELTO → CERRADO**. Un incidente RESUELTO puede reabrirse a EN_PROCESO. CERRADO es terminal.
RN-02	La asignación automática elige, entre los técnicos que atienden la categoría, al que tenga **menos incidentes abiertos** (no CERRADO); en caso de empate, el de menor id. Se registra en la bitácora.
RN-03	Solo el administrador asigna o reasigna, y el técnico elegido debe tener rol TECNICO y atender la categoría del incidente.
RN-04	Cambia el estado el administrador o el técnico asignado al incidente; ningún otro rol.

ID	Regla
RN-05	Visibilidad: administrador, todo; técnico, las categorías que atiende; solicitante, lo que reportó. Lo que está fuera del alcance responde 404.
RN-06	Un incidente CERRADO no admite comentarios ni nuevos cambios de estado.
RN-07	La identidad del autor (de un comentario, un evento o un incidente) la determina el token de sesión; los campos equivalentes del cuerpo de la petición se ignoran.
RN-08	**Abiertos** = estado distinto de CERRADO. **Críticos** = prioridad CRITICA aún no CERRADO.
RN-09	Las prioridades válidas son BAJA, MEDIA, ALTA, URGENTE y CRITICA; por defecto, BAJA.

4. Modelo de casos de uso

4.1 Diagrama de casos de uso

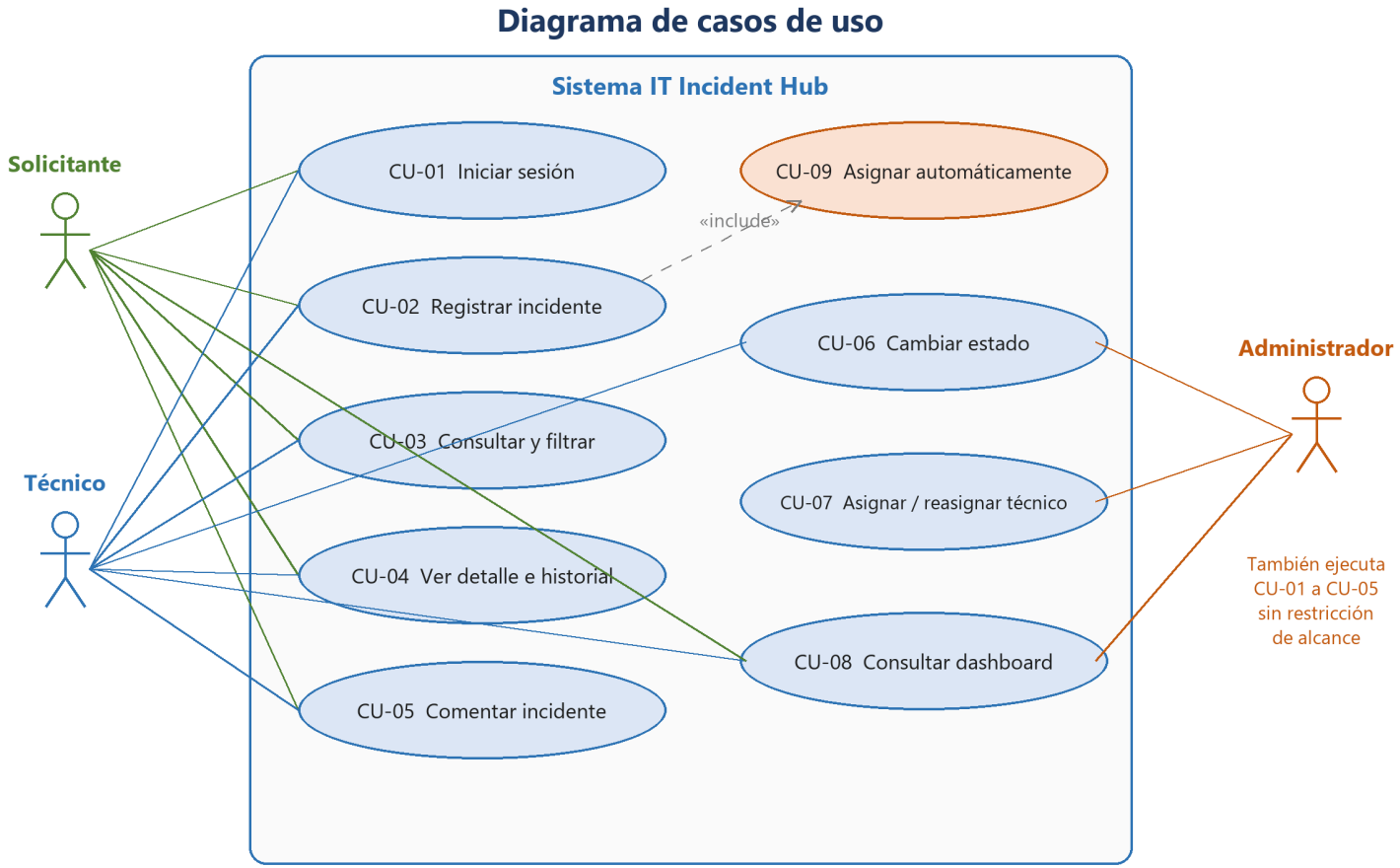


Figura 2. Casos de uso de IT Incident Hub por actor

4.2 Catálogo de casos de uso

Tabla 13. Catálogo de casos de uso

ID	Nombre	Actores	Requerimientos
CU-01	Iniciar y cerrar sesión	Solicitante, Técnico, Administrador	RF-01, RF-02
CU-02	Registrar incidente	Solicitante, Técnico, Administrador	RF-03, RF-13
CU-03	Consultar y filtrar incidentes	Todos	RF-05, RF-06
CU-04	Ver detalle e historial	Todos	RF-07, RF-11
CU-05	Comentar incidente	Todos	RF-10
CU-06	Cambiar estado	Técnico asignado, Administrador	RF-09, RF-14
CU-07	Asignar o reasignar técnico	Administrador	RF-08, RF-13
CU-08	Consultar dashboard	Todos (alcance según rol)	RF-12
CU-09	Asignar automáticamente	Sistema (incluido en CU-02)	RF-04

4.3 Especificación de los casos de uso principales

Tabla 14. CU-01 · Iniciar sesión

Actores	Solicitante, Técnico, Administrador
Precondición	El usuario existe y tiene contraseña registrada (hash)
Flujo principal	1. El usuario abre la aplicación y ve la pantalla de acceso. 2. Ingresa correo y contraseña y pulsa «Entrar». 3. El sistema valida las credenciales y entrega un token de 8 horas con los datos del usuario. 4. La aplicación guarda el token en `sessionStorage` y muestra la lista de incidentes de su alcance.
Flujos alternos	A1. Datos incompletos (correo o contraseña vacíos): el sistema responde 400. A2. Credenciales inválidas (correo inexistente o contraseña errónea): 401 con el mismo mensaje y el mismo costo de cálculo en ambos casos, para no revelar qué correos existen. A3. Token expirado durante el uso: cualquier petición responde 401 y la aplicación vuelve al login.
Postcondición	Sesión activa; las peticiones siguientes llevan `Authorization: Bearer <token>`
Reglas	RN-05, RN-07

Tabla 15. CU-02 · Registrar incidente (incluye CU-09)

Actores	Solicitante (principal), Técnico, Administrador
Precondición	Sesión activa

Flujo principal	1. El usuario pulsa «Nuevo incidente». 2. Completa título, descripción, categoría y, opcionalmente, prioridad. 3. El sistema valida los datos y crea el incidente a nombre del usuario de la sesión. 4. El sistema ejecuta CU-09: si la categoría tiene técnico, lo asigna al que tiene menos incidentes abiertos, cambia el estado a ASIGNADO y registra el evento. 5. La aplicación muestra «Incidente #n creado y asignado automáticamente a <técnico>».
Flujos alternos	A1. Datos inválidos (título vacío o mayor de 200, descripción vacía, categoría inexistente, prioridad no válida): 400 con la lista de errores. A2. La categoría no tiene técnico (por ejemplo «Otros»): el incidente queda NUEVO hasta que un administrador lo asigne.
Postcondición	Incidente persistido con estado NUEVO o ASIGNADO y, si hubo asignación, un evento ASIGNACION en la bitácora
Reglas	RN-02, RN-07, RN-09

Tabla 16. CU-06 · Cambiar estado

Actores	Técnico asignado, Administrador
Precondición	El incidente es visible para el actor y tiene técnico asignado
Flujo principal	1. El actor abre el detalle del incidente. 2. La aplicación ofrece solo los estados permitidos por el flujo (obtenidos de `GET /api/workflow`). 3. El actor elige el nuevo estado. 4. El sistema valida la transición, actualiza el incidente y registra un evento CAMBIO_ESTADO con «origen → destino».
Flujos alternos	A1. Sin permiso (solicitante o técnico no asignado): 403. A2. Transición inválida: 409 con la lista de estados permitidos. A3. Incidente sin técnico: 409 «el incidente debe tener un técnico asignado». A4. Incidente fuera del alcance: 404.
Postcondición	Estado actualizado, `updated_at` renovado y evento en la bitácora
Reglas	RN-01, RN-04, RN-06

Tabla 17. CU-07 · Asignar o reasignar técnico

Actores	Administrador
Precondición	Incidente en estado NUEVO, ASIGNADO o EN_PROCESO
Flujo principal	1. El administrador abre el detalle del incidente. 2. La aplicación lista solo los técnicos que atienden la categoría del incidente. 3. El administrador elige uno. 4. El sistema actualiza el técnico; si el incidente era NUEVO, pasa a ASIGNADO; registra un evento ASIGNACION («Asignado a…» o «Reasignado de…»).
Flujos alternos	A1. Actor sin rol de administrador: 403. A2. El usuario elegido no es técnico o no atiende la categoría: 400. A3. Incidente RESUELTO o CERRADO: 409.
Postcondición	Incidente con el nuevo técnico y evento registrado
Reglas	RN-03

Tabla 18. CU-05 · Comentar incidente

Actores	Solicitante, Técnico, Administrador
Precondición	El incidente es visible para el actor y no está CERRADO
Flujo principal	1. El actor escribe un comentario en el detalle del incidente. 2. El sistema valida que no esté vacío ni exceda 2000 caracteres. 3. Registra un evento COMENTARIO con el autor de la sesión.
Flujos alternos	A1. Comentario vacío o demasiado largo: 400. A2. Incidente CERRADO: 409. A3. Fuera de alcance: 404.
Postcondición	Comentario visible en la bitácora
Reglas	RN-06, RN-07

5. Arquitectura del sistema

5.1 Visión general y estilo arquitectónico

Se adoptó una arquitectura **cliente-servidor en tres capas** con una API REST sin estado entre la SPA y la base de datos. La separación permite probar cada capa de forma independiente y sustituir la interfaz o la base de datos sin tocar las reglas de negocio, que viven en el backend.

El diagrama de la Figura 1 (apartado 2.3) muestra las capas y los módulos. Este capítulo describe sus responsabilidades y las decisiones de diseño.

5.2 Backend

Tabla 19. Módulos del backend (`backend/src`)

Archivo	Responsabilidad
`app.js`	`createApp(db, { authSecret })`: monta rutas públicas (`/health`, `/api/auth`, `/api/workflow`), el middleware de sesión y las rutas protegidas. Recibe cualquier objeto con `query()` (Pool en producción, pg-mem en pruebas)
`server.js`	Arranque: lee `DATABASE_URL`, `PORT` y `AUTH_SECRET` (obligatoria con `NODE_ENV=production`)
`migrate.js`	Aplica en orden `database/migrations/*.sql` y registra cada una en `schema_migrations` (idempotente)
`auth.js`	Hash de contraseñas con scrypt y tokens HS256 firmados con HMAC, sin dependencias externas
`middleware/auth.js`	`requireAuth`: valida el token y carga al usuario (rol y categorías) desde la base en cada petición
`access.js`	Reglas de acceso por rol como funciones puras: `scopeFor`, `canSee`, `canOperate`, `canAssign`
`assignment.js`	Asignación automática: técnico de la categoría con menos incidentes abiertos
`workflow.js`	Fuente única del ciclo de vida: transiciones permitidas y estados asignables
`routes/*`	`auth`, `incidents`, `incident-actions`, `incident-history`, `stats`, `catalogs`, `workflow-route`

5.3 Frontend

Tabla 20. Componentes del frontend (`frontend/src`)

Componente	Función
`App`	Sesión: login, validación del token guardado, navegación Incidentes / Dashboard, usuario y cierre de sesión
`Login`	Pantalla de inicio de sesión
`session.js`, `api.js`	Token en `sessionStorage`; todas las llamadas HTTP con `Authorization`; vuelta al login si el backend responde 401

Componente	Función
`permissions.js`	Qué botones y textos mostrar según el rol (el backend es quien autoriza)
`IncidentList` + `Filters`	Lista y filtros; carga catálogos y reglas del flujo; texto del alcance según el rol
`IncidentForm`	Alta de incidentes con aviso de asignación automática
`IncidentDetail`	Asignación (administrador) y cambio de estado (administrador o técnico asignado)
`IncidentHistory`	Bitácora y comentarios
`Dashboard`	Tarjetas e indicadores con barras CSS accesibles, sin librerías de gráficos

5.4 Decisiones de diseño

Tabla 21. Decisiones de diseño y su motivo

Decisión	Motivo
Reglas del flujo en `workflow.js` y expuestas por `GET /api/workflow`	Una sola fuente de verdad; evita que frontend y backend diverjan (deuda D2, resuelta)
`createApp(db)` con la base inyectada	Probar la API sin un servidor de base de datos real
Autenticación propia con `crypto` de Node (scrypt + HS256)	Sin dependencias nuevas; menor superficie de ataque y de `npm audit`
Permisos aplicados en el backend; la interfaz solo oculta botones	La seguridad no puede depender del cliente
Fuera de alcance → 404; sin permiso → 403	No revelar la existencia de incidentes ajenos
Barras CSS en lugar de una librería de gráficos	Sin dependencias adicionales ni vulnerabilidades extra
Rama de integración `feature/login-roles`	Tres piezas interdependientes sin dejar `main` a medias
Merge commit en lugar de squash para `main`	Conservar la autoría de cada integrante en el historial
Un artefacto único promovido entre ambientes	Se prueba una vez y se despliega exactamente lo mismo

5.5 Vista de despliegue

Tabla 22. Nodos y puertos

Nodo	Contenido	Puerto
Navegador del usuario	SPA React (compilada o servida por Vite en desarrollo)	5173 (desarrollo)
Servidor de aplicación	API Express (Node 22)	3000
Base de datos	PostgreSQL 16 en contenedor Docker (`docker-compose.yml`)	5432
Agente autohospedado	Ejecuta el pipeline y deja los releases por ambiente en `_work/deploy/<ambiente>/`	-

Alcance del despliegue

En esta versión el despliegue por ambientes es **local**: copia versionada del artefacto en carpetas por ambiente sobre el agente autohospedado, con `current.json` y rollback. Sirve para demostrar promoción, aprobaciones y validación; no publica en un servidor externo. En un entorno real la API debe servirse por HTTPS.

6. Diseño de datos

6.1 Modelo entidad-relación

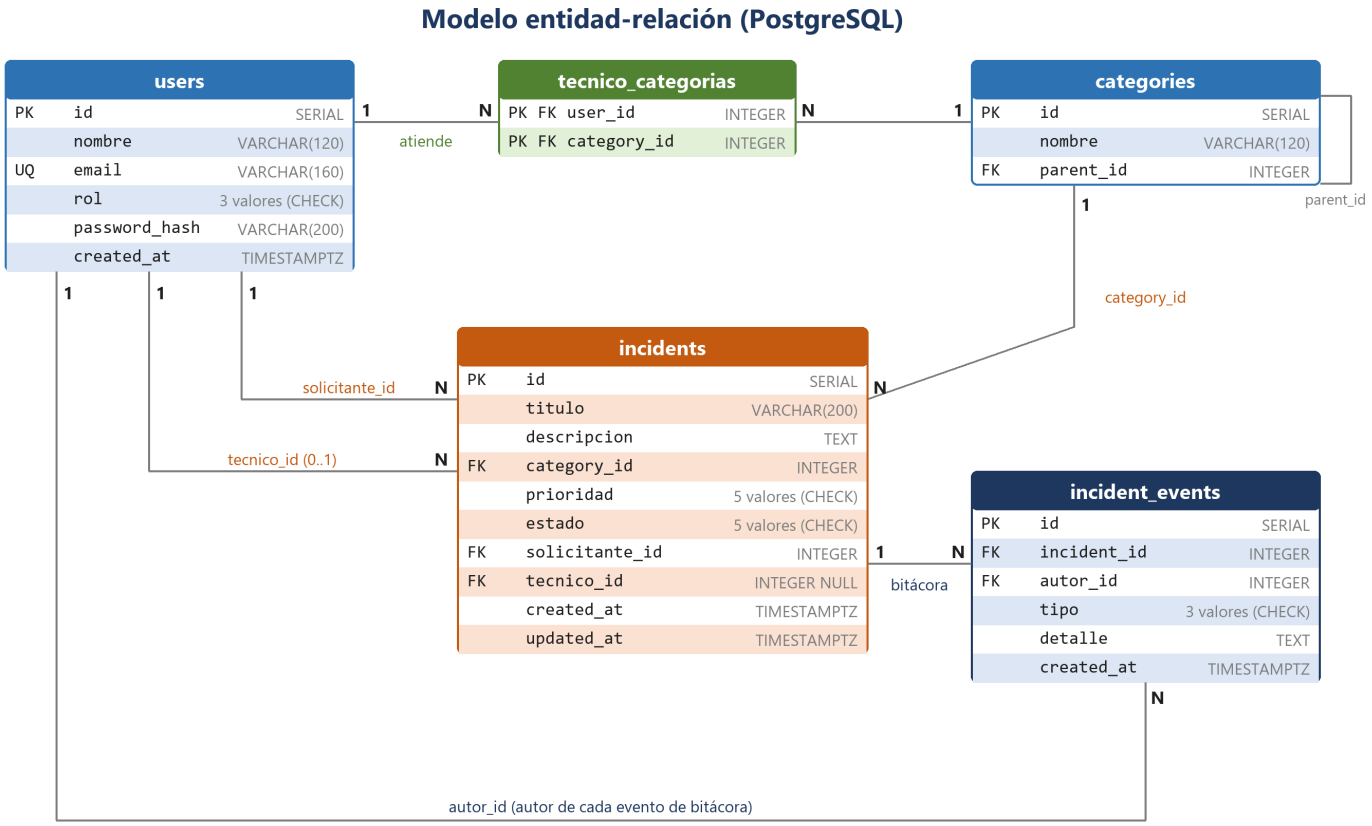


Figura 3. Modelo entidad-relación (PostgreSQL)

El modelo se define en `database/migrations/001_init.sql` (usuarios, categorías, incidentes y eventos) y `002_auth.sql` (credenciales y especialidades de los técnicos). La tabla `schema_migrations`, creada por `migrate.js`, registra qué migraciones se aplicaron.

6.2 Diccionario de datos

Tabla users

Tabla 23. Diccionario: users

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PK	Identificador
nombre	VARCHAR(120)	NOT NULL	Nombre completo
email	VARCHAR(160)	NOT NULL, UNIQUE	Correo (se compara en minúsculas al iniciar sesión)
rol	VARCHAR(20)	CHECK SOLICITANTE, TECNICO, ADMIN; por defecto SOLICITANTE	Perfil de acceso
password_hash	VARCHAR(200)	NULL permitido	Hash `scrypt$sal$hash`; NULL impide iniciar sesión
created_at	TIMESTAMPTZ	NOT NULL, por defecto now()	Fecha de alta

Tablas categories y tecnico_categorias

Tabla 24. Diccionario: categories y tecnico_categorias

Tabla.Columna	Tipo	Restricciones	Descripción
categories.id	SERIAL	PK	Identificador
categories.nombre	VARCHAR(120)	NOT NULL	Nombre de la categoría
categories.parent_id	INTEGER	FK → categories	Categoría padre (jerarquía; el filtro actual no incluye subcategorías, deuda D9)
tecnico_categorias.user_id	INTEGER	PK compuesta, FK → users	Técnico
tecnico_categorias.category_id	INTEGER	PK compuesta, FK → categories	Categoría que atiende; índice por `category_id`

Tabla incidents

Tabla 25. Diccionario: incidents

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PK	Número de incidente
titulo	VARCHAR(200)	NOT NULL	Resumen del problema
descripcion	TEXT	NOT NULL	Detalle del problema
category_id	INTEGER	NOT NULL, FK → categories	Categoría

Columna	Tipo	Restricciones	Descripción
prioridad	VARCHAR(10)	CHECK BAJA, MEDIA, ALTA, URGENTE, CRITICA; por defecto BAJA	Urgencia
estado	VARCHAR(12)	CHECK NUEVO, ASIGNADO, EN_PROCESO, RESUELTO, CERRADO; por defecto NUEVO	Estado del flujo
solicitante_id	INTEGER	NOT NULL, FK → users	Quien reportó (toma el valor del token)
tecnico_id	INTEGER	NULL, FK → users	Técnico asignado
created_at, updated_at	TIMESTAMPTZ	NOT NULL, por defecto now()	Alta y última modificación

Tabla incident_events

Tabla 26. Diccionario: incident_events (bitácora)

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PK	Identificador
incident_id	INTEGER	NOT NULL, FK → incidents	Incidente al que pertenece
autor_id	INTEGER	NOT NULL, FK → users	Autor (determinado por el token)
tipo	VARCHAR(20)	CHECK COMENTARIO, CAMBIO_ESTADO, ASIGNACION	Clase de evento
detalle	TEXT	NOT NULL	Texto del comentario, «origen → destino» o descripción de la asignación
created_at	TIMESTAMPTZ	NOT NULL, por defecto now()	Fecha del evento

6.3 Índices y datos de ejemplo

Hay índices por `estado`, `prioridad` y `tecnico_id` en incidentes, por `category_id` en las especialidades de técnicos y por `incident_id` en la bitácora, que respaldan los filtros y el historial. El archivo `database/seed.sql` carga seis usuarios de demostración (dos solicitantes, tres técnicos y un administrador), cinco categorías y las especialidades de los técnicos.

Tabla 27. Categorías y técnicos de la semilla

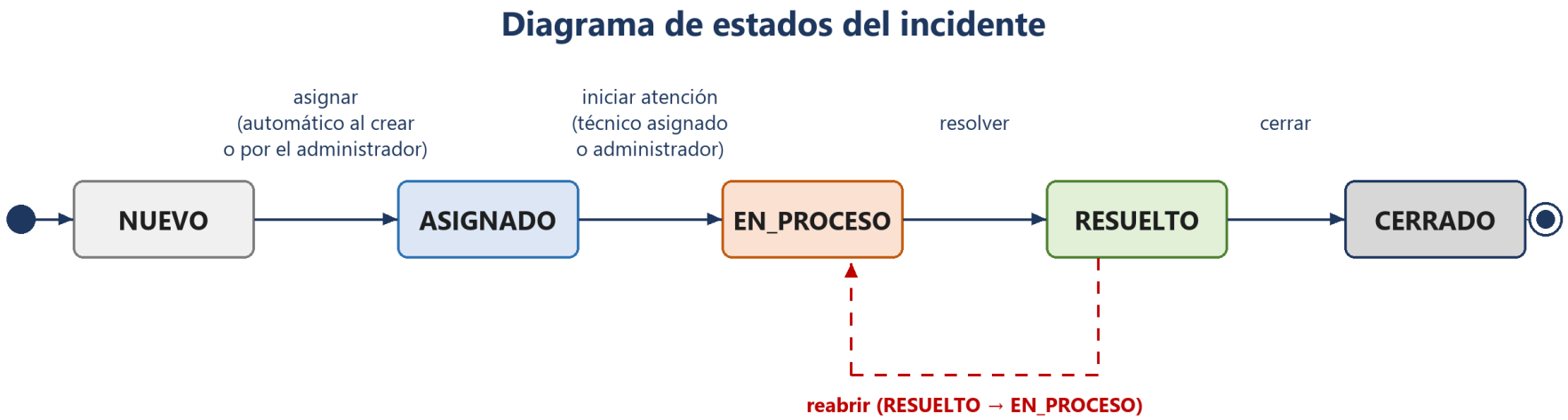
Categoría	Técnico que la atiende
1 · Equipo de cómputo	Luis Técnico
2 · Sistema Institucional	Marta Técnica
3 · Gestión de Accesos	Marta Técnica
4 · Telecomunicaciones	Carlos Teleco
5 · Otros	Ninguno (asignación manual por el administrador)

Credenciales de demostración

Todos los usuarios de la semilla comparten una **contraseña de prueba** documentada en el README (`IncidentHub#2026`); en el repositorio solo existe su hash scrypt. La semilla es solo para desarrollo y no debe usarse en un ambiente real.

7. Diseño del comportamiento

7.1 Ciclo de vida del incidente



Reglas: no se puede avanzar sin técnico asignado · se reasigna solo en NUEVO, ASIGNADO y EN_PROCESO, y solo a técnicos de la categoría · CERRADO es terminal y no admite comentarios · una transición inválida responde 409 con los estados permitidos.

Figura 4. Diagrama de estados del incidente

Tabla 28. Transiciones permitidas

Desde	Hacia	Quién	Condición
(creación)	NUEVO o ASIGNADO	Sistema	Queda ASIGNADO si la categoría tiene técnico (CU-09); si no, NUEVO
NUEVO	ASIGNADO	Administrador	Asignación manual a un técnico de la categoría
ASIGNADO	EN_PROCESO	Técnico asignado, Administrador	Debe tener técnico
EN_PROCESO	RESUELTO	Técnico asignado, Administrador	Debe tener técnico
RESUELTO	CERRADO	Técnico asignado, Administrador	Estado terminal
RESUELTO	EN_PROCESO	Técnico asignado, Administrador	Reapertura del incidente

Cualquier otra transición responde **409 Conflict** con la lista `permitidos`. La reasignación de técnico no cambia el estado, salvo en un incidente NUEVO, que pasa a ASIGNADO.

7.2 Secuencia: iniciar sesión y registrar un incidente

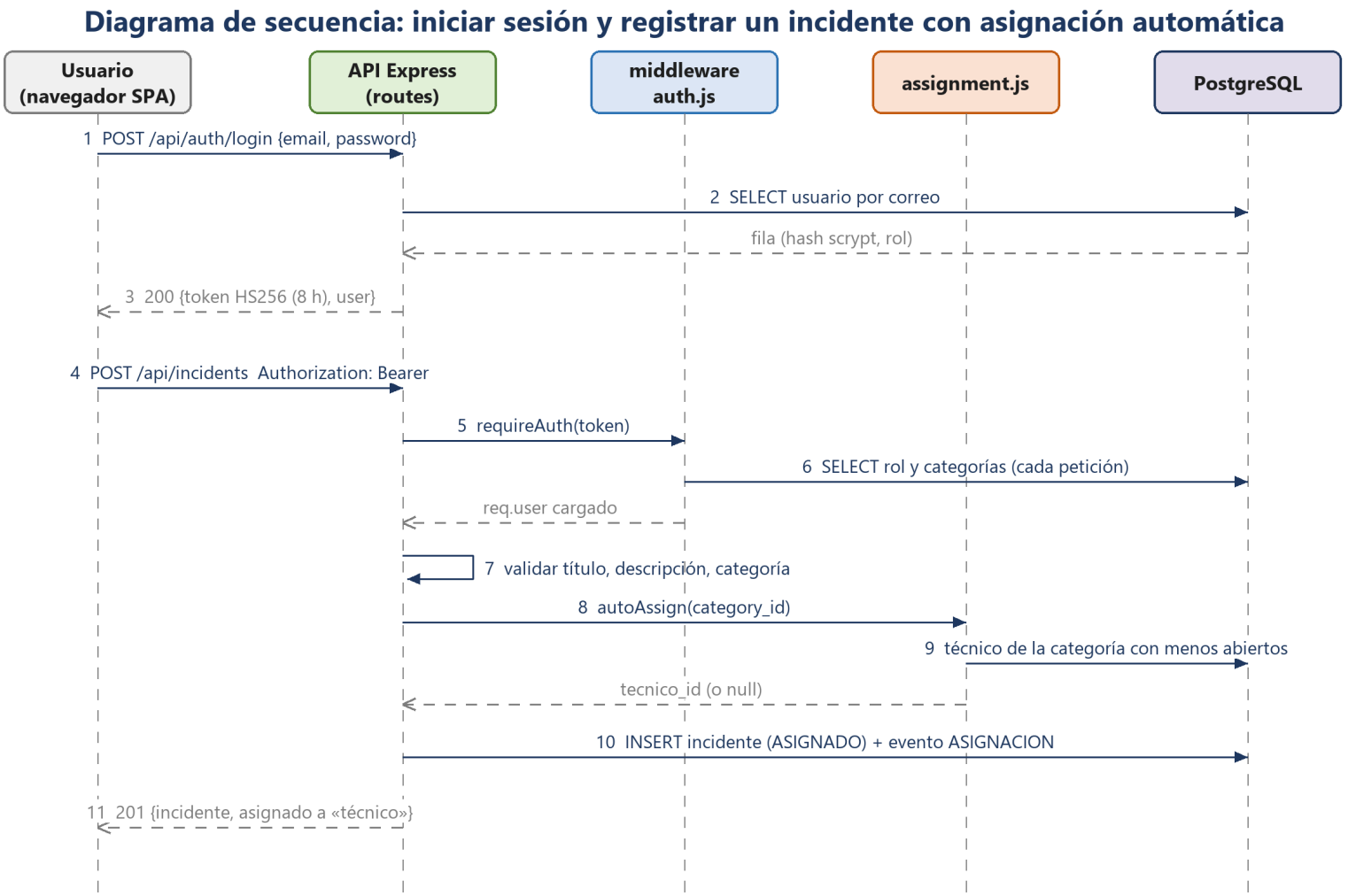


Figura 5. Secuencia de login y alta de un incidente con asignación automática

La secuencia ilustra dos decisiones de diseño importantes: el middleware consulta el rol y las categorías del usuario **en cada petición**, de modo que un cambio de permisos surte efecto de inmediato, y el autor del incidente se toma del token, nunca del cuerpo de la petición.

7.3 Algoritmo de asignación automática

```
autoAssign(incidente, creadorId):
  candidatos = técnicos que atienden incidente.category_id
  si candidatos está vacío:
      devolver incidente en estado NUEVO          // requiere asignación manual
  para cada técnico: abiertos = nº de incidentes con tecnico_id = técnico y estado <> CERRADO
  elegido = técnico con el menor valor de abiertos (empate: menor id)
  UPDATE incidents SET tecnico_id = elegido, estado = 'ASIGNADO'
  INSERT incident_events (tipo = 'ASIGNACION', autor = creadorId, detalle = 'Asignado automáticamente
a ...')
  devolver incidente actualizado
```

El criterio de «menos incidentes abiertos» reparte la carga entre técnicos de la misma especialidad. Un técnico de telecomunicaciones solo recibe y ve incidentes de esa categoría.

> **Limitación conocida (D3)**
> La actualización del incidente y la inserción del evento son dos consultas separadas, sin transacción. Si fallara la segunda, quedaría un incidente asignado sin su evento. Resolver esto exige pruebas contra PostgreSQL real (D4); ver el capítulo 15.

7.4 Cálculo del alcance de visibilidad

Tabla 29. Función `scopeFor(usuario)` (condición SQL añadida a las consultas)

Rol	Condición sobre `incidents`	Efecto
ADMIN	ninguna	Ve todos los incidentes
TECNICO	`category_id` ∈ categorías que atiende	Ve los de su especialidad, incluso los de otro técnico de esa categoría
SOLICITANTE	`solicitante_id` = usuario de la sesión	Ve solo los que reportó

La misma condición se aplica al listado, al detalle, al historial, a los comentarios y a las estadísticas, por lo que no existe una ruta que filtre información fuera del alcance. Un incidente ajeno responde 404, no 403, para no confirmar su existencia.

8. Especificación de la API REST

8.1 Convenciones

- Base: `http://localhost:3000`. Formato JSON en peticiones y respuestas.
- Todas las rutas, salvo `GET /health`, `POST /api/auth/login` y `GET /api/workflow`, exigen el encabezado `Authorization: Bearer <token>`.
- Los errores devuelven `{ "errors": [ "mensaje", ... ] }`; las transiciones inválidas añaden `permitidos`.
- Códigos usados: 200, 201, 400 (datos inválidos), 401 (sin sesión válida), 403 (sin permiso), 404 (no existe o fuera de alcance), 409 (conflicto con el estado) y 500.

8.2 Catálogo de endpoints

Tabla 30. Endpoints

Método	Ruta	Códigos	Descripción
GET	`/health`	200	Estado del servicio (público)
POST	`/api/auth/login`	200, 400, 401	`email`, `password` → `{ token, user }`
GET	`/api/auth/me`	200, 401	Usuario de la sesión con rol y categorías
POST	`/api/incidents`	201, 400, 401	`titulo`, `descripcion`, `category_id`, `prioridad` opcional; asigna automáticamente
GET	`/api/incidents`	200, 400, 401	Filtros `estado`, `prioridad`, `category_id`, `tecnico_id` (AND); más recientes primero; limitado por rol
GET	`/api/incidents/:id`	200, 400, 401, 404	Detalle
PATCH	`/api/incidents/:id/assign`	200, 400, 401, 403, 404, 409	Solo administrador; cuerpo `{ tecnico_id }`
PATCH	`/api/incidents/:id/status`	200, 400, 401, 403, 404, 409	Administrador o técnico asignado; cuerpo `{ estado }`
GET	`/api/incidents/:id/events`	200, 400, 401, 404	Bitácora cronológica; `?tipo=COMENTARIO|CAMBIO_ESTADO|ASIGNACION`
POST	`/api/incidents/:id/comments`	201, 400, 401, 404, 409	`{ comentario }` (máx. 2000)
GET	`/api/stats`	200, 401	`total`, `abiertos`, `cerrados`, `criticos`, `sin_asignar`, `por_estado`, `por_categoria`, `por_tecnico`
GET	`/api/categories`	200, 401	Catálogo de categorías
GET	`/api/users?rol=`	200, 401	Usuarios por rol (los técnicos incluyen `categorias`; sin correos ni hashes)
GET	`/api/workflow`	200	Reglas del flujo de estados (público)

8.3 Ejemplos

Inicio de sesión

```
POST /api/auth/login
{ "email": "ana@example.com", "password": "<contraseña de prueba>" }

200 OK
{ "token": "<token HS256>", "user": { "id": 1, "nombre": "Ana Solicitante", "rol": "SOLICITANTE", ...
} }

401 Unauthorized
{ "errors": [ "credenciales invalidas" ] }
```

Registrar un incidente

```
POST /api/incidents            Authorization: Bearer <token>
{ "titulo": "Sin internet en el piso 2", "descripcion": "Perdieron conexión desde las 14:00",
  "category_id": 4, "prioridad": "URGENTE" }

201 Created
{ "id": 8, "estado": "ASIGNADO", "tecnico_id": 5, "solicitante_id": 1, "category_id": 4, ... }
```

Transición inválida

```
PATCH /api/incidents/8/status  { "estado": "CERRADO" }     (el incidente está ASIGNADO)

409 Conflict
{ "errors": [ "transicion invalida: ASIGNADO -> CERRADO" ], "permitidos": [ "EN_PROCESO" ] }
```

Acceso fuera de alcance

```
GET /api/incidents/3     (usuario Luis, técnico de Equipo de cómputo; el incidente 3 es de
Telecomunicaciones)

404 Not Found
{ "errors": [ "incidente no encontrado" ] }
```

9. Seguridad

9.1 Principios

- **Ningún secreto en el repositorio, en capturas ni en documentación.** `.env` y `.env.*` están en `.gitignore`; solo se versiona `.env.example` con valores vacíos.
- **Mínimo privilegio:** cada rol ve y hace solo lo necesario, y se verifica en el servidor.
- **Defensa en profundidad:** validación de entrada, restricciones en la base, escáner de secretos y análisis de dependencias en cada corrida.
- **Sin confianza en el cliente:** la identidad sale del token; el cuerpo de la petición no puede suplantar a otro usuario.

9.2 Autenticación

Tabla 31. Mecanismos de autenticación

Aspecto	Implementación
Almacén de contraseñas	Solo el hash `scrypt$sal$hash` con sal aleatoria; comparación en tiempo constante
Respuesta ante error	Correo inexistente y contraseña errónea responden igual (401 `credenciales invalidas`) y con el mismo costo de cálculo (hash de relleno), para evitar enumeración de usuarios
Token	HS256 (HMAC-SHA256), vigencia de 8 horas; se rechazan firmas distintas, alteraciones, expiración, formato incorrecto y `alg: none`
Secreto de firma	`AUTH_SECRET`; obligatorio con `NODE_ENV=production`. En desarrollo, si falta, se genera uno aleatorio por proceso y el servidor avisa
Datos de la sesión	Rol y categorías se leen de la base en cada petición: revocar un permiso aplica al instante

9.3 Autorización: matriz de permisos

Tabla 32. Matriz de permisos por rol

Acción	Solicitante	Técnico	Administrador
Ver incidentes	Solo los que reportó	Los de las categorías que atiende	Todos
Crear incidente	Sí (se asigna solo)	Sí	Sí
Comentar	En los que ve	En los que ve	En los que ve
Asignar o reasignar técnico	No (403)	No (403)	Sí, a un técnico de la categoría
Cambiar estado	No (403)	Solo los asignados a él	Sí
Dashboard	De sus incidentes	De sus categorías	Global

9.4 Modelo de amenazas resumido

Tabla 33. Amenazas y controles

Amenaza	Control aplicado	Estado
Suplantación de otro usuario enviando `autor_id` o `solicitante_id`	Se ignoran los campos del cuerpo; la identidad sale del token (RN-07)	Mitigada
Acceso a incidentes ajenos (referencia directa a objetos)	Alcance por rol en todas las consultas; 404 fuera de alcance	Mitigada
Escalada de privilegios	Verificación de rol en el servidor; rol leído de la base en cada petición	Mitigada
Falsificación o alteración del token	Firma HMAC verificada; `alg: none` rechazado; vigencia limitada	Mitigada
Enumeración de usuarios en el login	Respuesta y costo idénticos para correo inexistente y contraseña errónea	Mitigada
Fuga de secretos en el repositorio	Escáner `check-secrets.js` en cada corrida; `.gitignore`; variable group y GitHub Secrets	Mitigada
Dependencias vulnerables	`npm audit` en cada corrida con falla en high/critical	Mitigada
Fuerza bruta contra contraseñas	No hay límite de intentos ni bloqueo de cuentas	**Abierta (D10)**
Robo de token	Sin revocación; token en `sessionStorage` (accesible ante un XSS); vigencia de 8 horas	**Abierta (D11)**
Interceptación de tráfico	La API debe publicarse por HTTPS; el entorno actual es local	**Pendiente en producción real**

9.5 Gestión de secretos

Tabla 34. Inventario de secretos

Secreto	Uso	Dónde se guarda
`DB_PASSWORD`	Contraseña de PostgreSQL (`docker-compose.yml`, `DATABASE_URL`)	Variable group protegido `incident-hub-secrets` (Azure) y GitHub Secret
`AUTH_SECRET`	Firma de los tokens de sesión	Entorno de ejecución del servidor; no forma parte del CI
PAT del agente	Registrar el agente autohospedado	Solo se escribe en la consola al configurar el agente; se revoca después

Una variable secreta de Azure no llega sola a los scripts: cada paso la mapea con `env:` y Azure enmascara el valor en los registros. `devops/scripts/verify-secret.js` confirma que el secreto llegó **sin imprimir su valor**. Si un secreto se expone, el procedimiento es rotarlo de inmediato, actualizar las fuentes, revocar el valor expuesto (borrar el commit no basta) y registrar un Work Item.

9.6 Controles automáticos

Tabla 35. Controles automáticos de seguridad

Control	Dónde	Qué evita
`check-secrets.js`	Azure Pipelines y GitHub Actions	Claves privadas, tokens de GitHub, claves AWS, PAT de Azure DevOps, credenciales en URL y asignaciones literales de password/token. Reporta archivo y línea, nunca el valor
`tests/secrets.test.js`	Ambos CI	Verifica las reglas del escáner con valores falsos armados en tiempo de ejecución
`permissions.test.js`, `access.test.js`	Ambos CI	Matriz de permisos por rol: visibilidad, 403 y 404
`auth.test.js`, `auth-routes.test.js`	Ambos CI	Hash, tokens y login sin enumeración de usuarios
`npm audit --audit-level=high`	Ambos CI	Publicar con dependencias vulnerables
`docker-compose.yml`	Local	`DB_PASSWORD` obligatoria (`:?`): no hay contraseña por defecto

> **Alcance del escáner**
> El escáner de secretos es un control propio y sencillo: revisa los archivos versionados actuales, no el historial de Git, y no sustituye a una herramienta dedicada como gitleaks.

10. DevOps y gestión del ciclo de vida

10.1 Trazabilidad de extremo a extremo

El proceso exige que cada cambio pueda seguirse desde su requerimiento hasta el entorno donde corre: **Work Item (AB#ID) → rama → commit con AB#ID → Pull Request → corrida de pipeline → artefacto → versión (tag) → release por ambiente.**

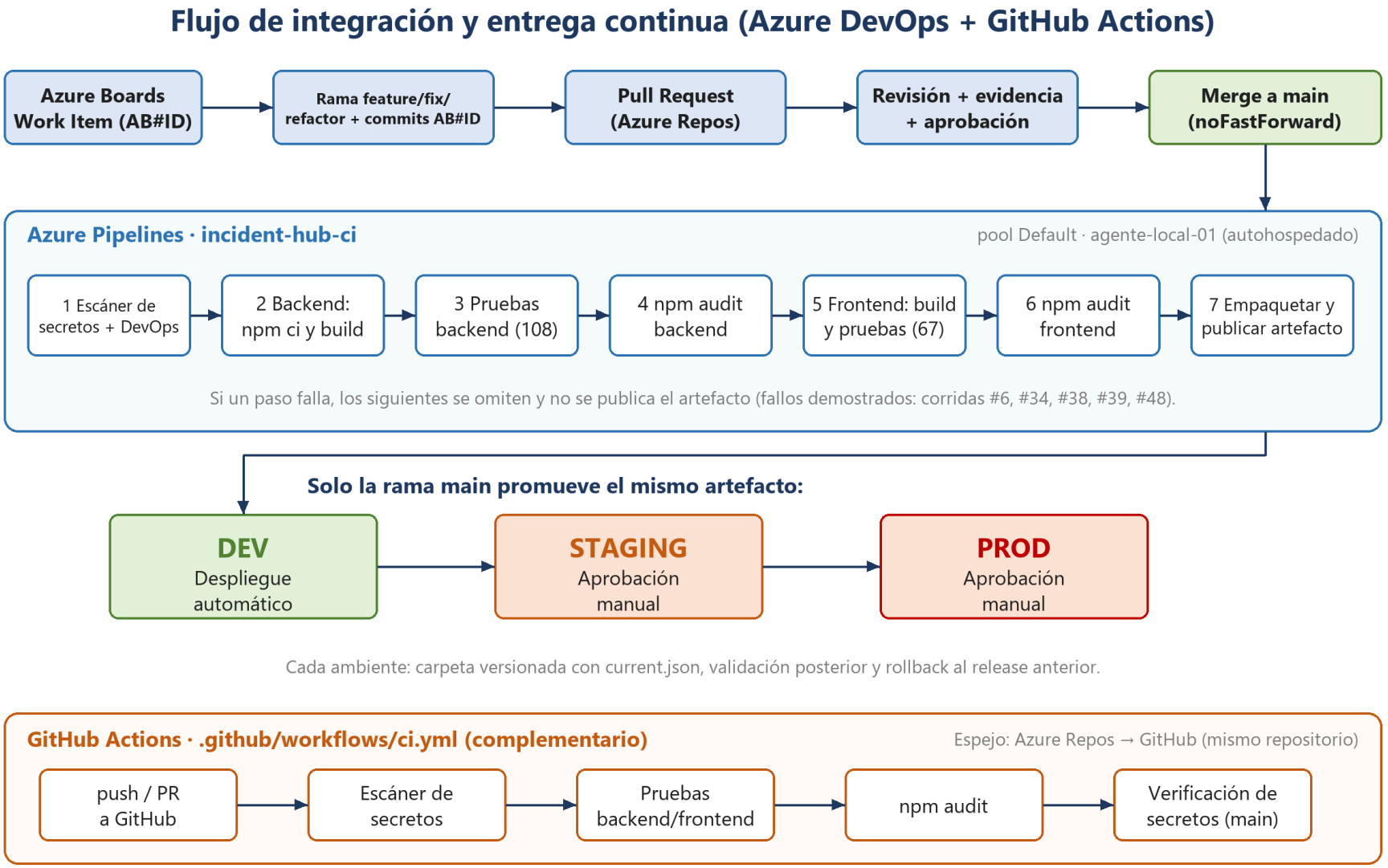


Figura 6. Flujo de integración y entrega continua

10.2 Gestión del trabajo en Azure Boards

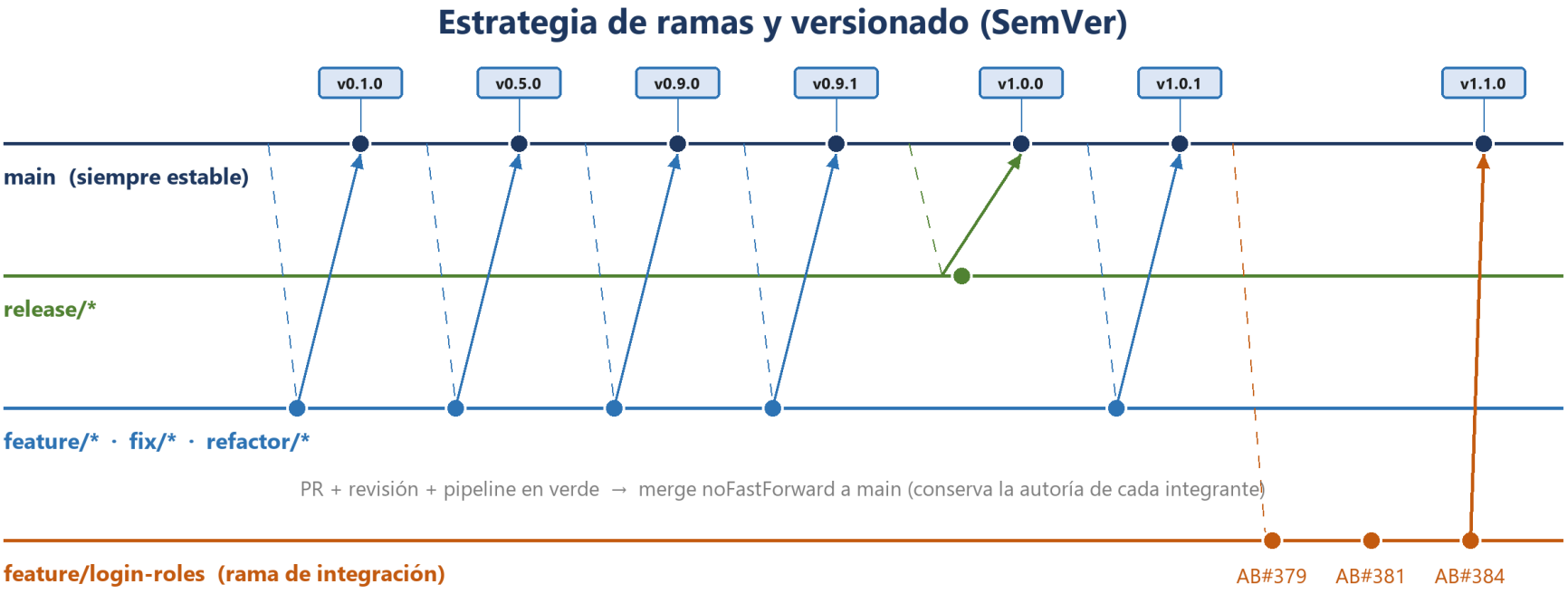
El backlog se gestiona con el proceso **Agile** de Azure Boards, con estimaciones en horas (Tasks) y puntos (Stories) y roles asignados mediante etiquetas.

Tabla 36. Composición del backlog

Tipo de elemento	Cantidad	Ejemplos
Epic	2	Plataforma de gestión de incidentes · Entrega continua DevOps
Feature	5	Registro y consulta · Operación, seguimiento e indicadores · Pipeline CI · Seguridad y lanzamiento · Autenticación y roles (AB#375)
User Story	12	Crear incidente · Consultar y filtrar · Asignar y cambiar estado · Comentarios y bitácora · Dashboard · Pipeline CI · Seguridad · Versionado y release · Deuda técnica
Task	40	Modelo de datos · Endpoints · UI · Pipeline · Secretos · Ambientes · Login y roles · Pruebas
Bug	3	AB#358 orden del listado · AB#371 ZIP de builds anteriores · AB#374 vulnerabilidad en body-parser

La exportación completa de los 62 elementos está en `docs/evidencia/backlog-azure-boards.csv`.

10.3 Estrategia de ramas y Pull Requests



Convención: ramas feature/<AB#ID>-descripción · commits «feat: … AB#ID» · tags anotados vMAJOR.MINOR.PATCH sobre main (13 tags hasta v1.1.0).

Figura 7. Estrategia de ramas y versionado

Tabla 37. Reglas de trabajo con Git

Elemento	Regla
`main`	Siempre estable; solo recibe cambios mediante Pull Request con revisión y pipeline en verde
Ramas de trabajo	`feature/<AB#ID>-descripcion`, `fix/<AB#ID>-descripcion`, `refactor/<AB#ID>-descripcion`
Rama de integración	`feature/login-roles` agrupó las tres piezas interdependientes de la v1.1.0 antes de llegar a `main`
Commits	Mensaje convencional con el Work Item, por ejemplo `feat: crear incidente AB#12`
Pull Request	Descripción, Work Item vinculado, comentario con la evidencia de la corrida y voto de aprobación
Fusión	Merge commit (`noFastForward`) para conservar la autoría de cada integrante; las ramas no se borran

Hasta la fecha se han abierto 32 Pull Requests en Azure Repos (`docs/evidencia/pull-requests.csv`). **Limitación:** al ser un proyecto individual, la misma identidad de Azure DevOps aprueba y fusiona; no existe un revisor independiente.

10.4 Integración continua

Tabla 38. Etapas del pipeline `incident-hub-ci` (Azure Pipelines, agente autohospedado)

Paso	Acción	Falla si…
1	Checkout e instalación de Node 22 (`UseNode@1`)	No se puede preparar el entorno
2	Escáner de secretos y pruebas de utilidades DevOps (27)	Se detecta un secreto o falla una prueba
3	Verificación del secreto `DB_PASSWORD` del variable group	El secreto no llega al paso
4	Backend: `npm ci`, build y pruebas (108), publicación de resultados	Falla una prueba
5	Backend: `npm audit --audit-level=high`	Hay vulnerabilidades high o critical
6	Frontend: `npm ci`, build, pruebas (67) y auditoría	Falla build, prueba o auditoría
7	Empaquetado de backend y frontend (ZIP) y publicación del artefacto `incident-hub`	No se generó el paquete

GitHub Actions (`.github/workflows/ci.yml`) ejecuta de forma complementaria, en cada push y Pull Request a GitHub, tres trabajos: escáner de secretos con pruebas de utilidades, pruebas y auditoría del backend, y pruebas, build y auditoría del frontend. La verificación del secreto `DB_PASSWORD` se hace solo en pushes a `main`, porque los secretos no se entregan a Pull Requests de forks.

El agente autohospedado `agente-local-01` (pool `Default`) es una máquina local; su instalación y registro con un PAT se describen en `devops/README.md`. Al ser un único agente, las corridas se encolan.

10.5 Fallos controlados del pipeline

Se provocaron y documentaron fallos reales para demostrar que los controles actúan y que el equipo sabe diagnosticarlos (detalle en `docs/ci-failure-diagnosis.md`).

Tabla 39. Fallos del pipeline, causa y corrección

Corrida	Fallo	Causa raíz	Corrección
#6	Prueba de orden del listado	Regresión introducida a propósito: `ORDEN_LISTADO` ascendente	Bug AB#358: orden descendente por fecha e id; corrida #7 en verde
#34	Despliegue a dev	La carpeta del artefacto conservaba ZIP de corridas anteriores (hallaba 2)	Bug AB#371: usar el ZIP exacto de la build (PR #22)
#38, #39	Revisión de dependencias	Aviso nuevo de `body-parser` < 1.20.9 (GHSA-27r3-jxmh-p828, alta)	Bug AB#374: `npm audit fix` actualiza a 1.20.9 (PR #25); corrida #40 en verde
#48	Escáner de secretos	Pruebas del frontend con literales con forma de token y contraseña	Valores armados en tiempo de ejecución; corrida #50 en verde

10.6 Entrega por ambientes y release

Tabla 40. Ambientes

Ambiente	Environment en Azure	Disparo	Control
dev	`incident-hub-dev`	Automático tras CI verde en `main`	Validación posterior con `deploy.js estado`
staging	`incident-hub-staging`	Tras un dev correcto	**Aprobación manual** (check *Approval*)
prod	`incident-hub-prod`	Tras un staging correcto	**Aprobación manual** (check *Approval*)

- Solo la rama `main` ejecuta los *stages* de despliegue; las ramas de trabajo ejecutan únicamente el CI.
- Cada despliegue deja un release versionado en `_work/deploy/<ambiente>/<versión>-b<build>/` con `backend/`, `frontend/` y `release.json` (versión, build, commit y fecha). `current.json` apunta al release vigente y al anterior.
- El mismo artefacto se promueve entre ambientes: se prueba una vez y se despliega lo mismo.
- La corrida final de `main` (20261026.5) fue aprobada por el responsable del proyecto y completó dev → staging → producción en verde.

Rollback sin recompilar: intercambia el release vigente por el anterior y se puede revertir a su vez.

```
node devops/scripts/deploy.js rollback --entorno <dev|staging|prod> --raiz <agente>/_work/deploy
node devops/scripts/deploy.js estado   --entorno <dev|staging|prod> --raiz <agente>/_work/deploy
```

Tabla 41. Lista de verificación de un release

Paso	Verificación
1	Pull Requests del hito fusionados y Work Items cerrados
2	Pipeline de `main` y GitHub Actions en verde

Paso	Verificación
3	`npm audit` sin high/critical, o decisión registrada en `docs/dependencies.md`
4	Deudas nuevas registradas en `docs/technical-debt.md`
5	Despliegue en dev validado
6	Aprobación a staging y pruebas manuales de humo
7	Aprobación a producción y validación posterior
8	Tag anotado `vX.Y.Z` y notas en `CHANGELOG.md`

> **Alcance real**
> Al ser un proyecto individual, quien aprueba es la misma persona que ejecuta el pipeline; la aprobación queda registrada con fecha en el historial de la corrida. Está pendiente reforzar los ambientes con el check *Branch control* (solo `main`).

10.7 Versionado semántico

Backend y frontend comparten versión. Las versiones siguen **SemVer**: cambios incompatibles en MAJOR, funcionalidad nueva en MINOR y correcciones en PATCH. Los tags `vX.Y.Z` son anotados y apuntan al commit de `main` que cierra cada hito; una prueba automática verifica la coherencia de versiones entre paquetes y `CHANGELOG.md`.

Tabla 42. Versiones publicadas

Versión	Fecha	Hito
v0.1.0	2026-10-17	Estructura, backlog, modelo de datos y API de incidentes con filtros
v0.2.0 / v0.2.1	2026-10-17	CI en Azure y GitHub; corrección con fallo controlado (Bug AB#358)
v0.3.0	2026-10-17	Frontend: lista y filtros
v0.4.0	2026-10-17	Asignación, transiciones de estado y bitácora
v0.5.0	2026-10-18	Funcionalidad completa: dashboard, historial, formulario de alta
v0.6.0	2026-10-18	Deuda técnica: D1 y D2 resueltas
v0.7.0	2026-10-18	Secretos protegidos y escáner
v0.8.0	2026-10-18	Ambientes y aprobaciones
v0.9.0 / v0.9.1	2026-10-20	Versionado unificado; corrección del despliegue (Bug AB#371)
v1.0.0	2026-10-24	Primera versión estable
v1.1.0	2026-10-25	Inicio de sesión con roles y asignación automática por categoría

10.8 Análisis de dependencias

Tabla 43. Hallazgos de `npm audit`

Fecha	Componente	Hallazgo	Severidad	Decisión
2026-10-17	frontend (dev)	vite 5: path traversal y NTLMv2 vía launch-editor	high	Corregida: vite 8
2026-10-17	frontend (dev)	vitest 2 / tinypool: prototype pollution a RCE	critical	Corregida: vitest 5
2026-10-17	frontend (dev)	esbuild y transitivas de vite	moderate	Corregida con vite 8 y vitest 5
2026-10-24	backend (producción)	body-parser 1.20.8: denegación de servicio (GHSA-27r3-jxmh-p828)	high	Corregida: 1.20.9 (Bug AB#374)

Estado actual: `npm audit` reporta **0 vulnerabilidades** en backend y frontend. Los avisos aparecen con el tiempo, por eso el análisis se ejecuta en cada corrida y no solo al instalar; el aviso de `body-parser` apareció después de varias corridas en verde y detuvo el pipeline, tal como se diseñó.

11. Aseguramiento de la calidad

11.1 Estrategia de pruebas

Tabla 44. Niveles y herramientas de prueba

Nivel	Herramienta	Qué cubre	Pruebas
API y reglas	`node --test` + supertest + pg-mem	Rutas, validaciones, transiciones, bitácora, indicadores, tokens, login, matriz de permisos y asignación automática	108
Interfaz	Vitest + Testing Library	Componentes, filtros, formularios, login, sesión y expiración, permisos por rol y errores	67
DevOps	`node --test`	Escáner de secretos, despliegue y rollback, coherencia de versiones	27
Integración continua	Azure Pipelines y GitHub Actions	Todo lo anterior más `npm audit`	202 + auditoría

> **Límite conocido (D4)**
> pg-mem simula PostgreSQL en memoria pero no reproduce todo su comportamiento (por ejemplo, `ROLLBACK` no revierte), por lo que no se prueban transacciones. Las instrucciones de Docker con PostgreSQL real no se han verificado de punta a punta.

11.2 Plan de pruebas de aceptación

Los casos siguientes validan los requerimientos desde la perspectiva del usuario. La columna **Cobertura automática** indica qué suite los ejercita; el caso CP-02 se ejecutó además de forma manual en la demostración local.

Tabla 45. Casos de prueba de aceptación

ID	Escenario y pasos	Resultado esperado	Cobertura automática
CP-01	Iniciar sesión con credenciales válidas	Se muestra la lista de incidentes del alcance del rol	`auth-routes`, `Login`
CP-02	Como Ana (solicitante), crear un incidente de Telecomunicaciones	Mensaje «creado y asignado automáticamente a Carlos Teleco»; estado ASIGNADO	`auto-assign`, `IncidentForm`
CP-03	Iniciar sesión con contraseña errónea	Mensaje de credenciales inválidas; sin sesión	`auth-routes`, `Login`
CP-04	Como Luis (técnico de cómputo), abrir el incidente de telecomunicaciones	No aparece en su lista; el acceso directo devuelve 404	`permissions`, `access`
CP-05	Como Carlos, cambiar el estado a EN_PROCESO y comentar	Estado actualizado y dos eventos en la bitácora	`incident-actions`, `incident-history`
CP-06	Como Ana, intentar cambiar un estado (llamada directa a la API)	403 sin permiso	`permissions`

ID	Escenario y pasos	Resultado esperado	Cobertura automática
CP-07	Como administrador, reasignar a un técnico que no atiende la categoría	400 «el técnico no atiende la categoría»	`incident-actions`
CP-08	Avanzar de ASIGNADO a CERRADO	409 con los estados permitidos	`workflow`, `incident-actions`
CP-09	Crear un incidente de «Otros»	Queda NUEVO sin técnico hasta que el administrador lo asigne	`auto-assign`
CP-10	Filtrar por estado y prioridad a la vez	Solo incidentes que cumplen ambos	`incident-filters`, `IncidentList`
CP-11	Abrir el dashboard como técnico	Indicadores limitados a sus categorías	`stats`, `Dashboard`
CP-12	Token expirado durante el uso	La aplicación vuelve al login	`api`, `App`

11.3 Resultados y métricas

Tabla 46. Indicadores de calidad

Indicador	Valor
Pruebas automáticas	202 (backend 108, frontend 67, DevOps 27), todas en verde
Vulnerabilidades (`npm audit`)	0 en backend y en frontend
Escáner de secretos	Sin hallazgos en archivos versionados
Corridas de Azure Pipelines	Más de 50; fallos controlados y corregidos: #6, #34, #38, #39 y #48
Corridas de GitHub Actions	20 registradas, incluida una disparada por un Pull Request de GitHub
Deuda técnica	11 identificadas, 3 resueltas (D1, D2, D5)

12. Manual de instalación y operación

12.1 Requisitos

Tabla 47. Requisitos previos

Componente	Versión	Uso
Node.js y npm	22 (o 20.19+)	Backend, frontend y pruebas
Git	Reciente	Clonar el repositorio
Docker Desktop	Reciente	Solo para PostgreSQL local (no necesario en el modo demostración)
PowerShell	5.1 o superior	Los comandos de esta guía

12.2 Instalación con PostgreSQL (Docker)

1. **Clonar e instalar dependencias:** `git clone https://github.com/SotoJ08/incident-hub.git`, luego `npm ci` dentro de `backend` y de `frontend`.
2. **Levantar la base de datos:** definir una contraseña local en `$env:DB_PASSWORD` (no se guarda en el repositorio) y ejecutar `docker compose up -d db`.
3. **Migrar y cargar datos de ejemplo:** definir `DATABASE_URL`, ejecutar `npm run migrate` en `backend` y aplicar `database/seed.sql` con `psql`.
4. **Iniciar la API (puerto 3000):** `npm start` en `backend`; comprobar `http://localhost:3000/health`.
5. **Iniciar la interfaz (puerto 5173):** `npm run dev` en `frontend`; abrir `http://localhost:5173`.

```
$env:DB_PASSWORD = "<su-contraseña-local>"
docker compose up -d db
$env:DATABASE_URL = "postgres://incident:$($env:DB_PASSWORD)@localhost:5432/incident_hub"
cd backend; npm run migrate; cd ..
Get-Content database/seed.sql | docker compose exec -T db psql -U incident -d incident_hub
cd backend; npm start            # terminal 1
cd frontend; npm run dev         # terminal 2
```

12.3 Modo demostración sin Docker

El paquete de entrega incluye `LOCAL-DEMO/`, un servidor que usa la **misma API y la misma interfaz** con una base de datos en memoria precargada con siete incidentes de ejemplo coherentes con los roles. Solo requiere Node.js.

```
cd incident-hub
.\LOCAL-DEMO\start-demo.ps1
# o manualmente, en dos terminales:
node LOCAL-DEMO/dev-memory.js          # API en :3000
cd frontend; npm run dev               # interfaz en :5173
```

Los cambios se pierden al detener la demostración. Los usuarios y la contraseña de prueba están en el apartado 13.1.

12.4 Variables de entorno

Tabla 48. Variables de entorno

Variable	Obligatoria	Descripción
`DATABASE_URL`	Sí (backend)	Cadena de conexión a PostgreSQL
`DB_PASSWORD`	Sí (Docker)	Contraseña de la base; `docker-compose.yml` no define valor por defecto
`DB_USER`	No	Usuario de la base (por defecto `incident`)
`PORT`	No	Puerto de la API (por defecto 3000)
`AUTH_SECRET`	Sí con `NODE_ENV=production`	Secreto de firma de tokens; en desarrollo se genera uno aleatorio por proceso
`NODE_ENV`	No	Con `production` se exige `AUTH_SECRET`

12.5 Ejecución de pruebas y verificaciones

```
cd backend;  npm test                # 108 pruebas de API (pg-mem)
cd frontend; npm test                # 67 pruebas de interfaz
node --test "tests/*.test.js"        # 27 pruebas de utilidades DevOps (requiere el repositorio Git)
node devops/scripts/check-secrets.js  # escáner de secretos
```

12.6 Operación

Tabla 49. Tareas operativas frecuentes

Tarea	Procedimiento
Aplicar nuevas migraciones	`npm run migrate` en `backend`; es idempotente y registra en `schema_migrations`
Agregar un técnico o una especialidad	Insertar en `users` y `tecnico_categorias` (no hay pantalla de administración de usuarios todavía)
Regenerar un hash de contraseña	`node backend/scripts/hash-password.js "<contraseña>" <sal-hex>`
Volver a la versión anterior	`deploy.js rollback` (apartado 10.6)
Rotar `AUTH_SECRET`	Cambiar el valor y reiniciar la API; invalida todas las sesiones
Registrar el agente de Pipelines	Guía en `devops/README.md`; el PAT se escribe solo en la consola y se revoca después

12.7 Solución de problemas

Tabla 50. Problemas comunes

Síntoma	Causa probable y solución
El puerto 3000 o 5173 está en uso	Hay otra instancia en ejecución; ciérrela o reinicie el equipo
`node` no se reconoce	Instale Node.js y abra una PowerShell nueva

Síntoma	Causa probable y solución
La pantalla de acceso no deja entrar	Compruebe que la API siga abierta (`/health` debe responder `{"status":"ok"}`); al reiniciar la API en desarrollo las sesiones se pierden
`docker compose` indica que `DB_PASSWORD` es obligatoria	Defina `$env:DB_PASSWORD` en la misma terminal antes de ejecutar el comando
Las pruebas de `tests/` fallan con «not a git repository»	Ejecútelas dentro de un clon del repositorio (o del bundle Git incluido en la entrega)

13. Manual de usuario

13.1 Acceso al sistema

Abra `http://localhost:5173` e ingrese con su correo y contraseña. La sesión dura 8 horas y se cierra con el botón «Cerrar sesión». Para la demostración existen estos usuarios, todos con la contraseña de prueba `IncidentHub#2026`:

Tabla 51. Usuarios de demostración

Correo	Rol	Qué ve y qué hace
`ana@example.com`, `pedro@example.com`	Solicitante	Crea incidentes y ve solo los suyos; puede comentar
`luis@example.com`	Técnico	Atiende Equipo de cómputo
`marta@example.com`	Técnico	Atiende Sistema Institucional y Gestión de Accesos
`carlos@example.com`	Técnico	Atiende Telecomunicaciones (solo ve temas de telecomunicaciones)
`admin@example.com`	Administrador	Ve todo, asigna y reasigna técnicos, cambia cualquier estado



Figura 8. Pantalla de inicio de sesión

13.2 Solicitante: reportar y seguir un incidente

1. Inicie sesión; verá «Sus incidentes», con la lista de los que reportó.
2. Pulse **Nuevo incidente**, escriba el título y la descripción, elija la categoría y la prioridad, y pulse **Crear incidente**.
3. El sistema confirma el número de incidente y, si hay técnico para esa categoría, le informa a quién se asignó automáticamente.
4. Use **Ver** para abrir el detalle, leer la bitácora y agregar comentarios mientras el incidente no esté cerrado.
5. Use los filtros (estado, prioridad, categoría, técnico) para encontrar un incidente.



Figura 9. Lista de incidentes de un solicitante



Figura 10. Incidente de Telecomunicaciones creado y asignado automáticamente

13.3 Técnico: atender incidentes

1. Inicie sesión: verá los incidentes de las categorías que atiende (el técnico de telecomunicaciones solo ve esa categoría).
2. Abra un incidente asignado a usted con **Ver**.
3. Cambie el estado siguiendo el flujo: ASIGNADO → EN_PROCESO → RESUELTO → CERRADO. Si el usuario reporta que el problema persiste, puede reabrir un incidente RESUELTO a EN_PROCESO.
4. Documente cada avance con un comentario; queda en la bitácora con su nombre y la fecha.

> **Qué no puede hacer un técnico**
> No puede asignar ni reasignar técnicos, ni cambiar el estado de incidentes asignados a otro técnico. La interfaz oculta esas acciones y el servidor las rechaza con 403 si se intentan.

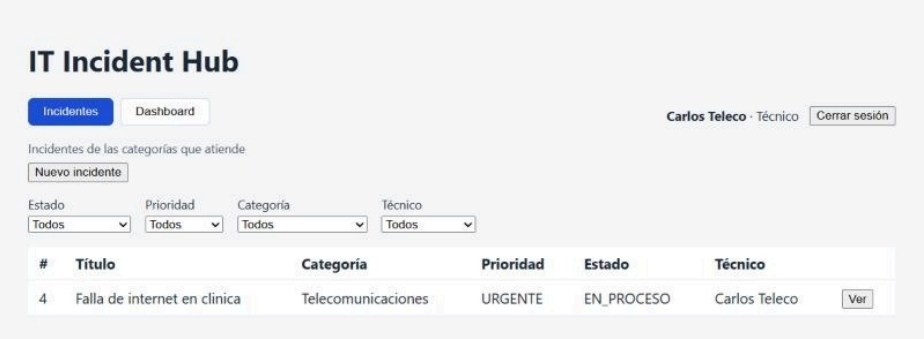


Figura 11. Vista del técnico de telecomunicaciones

13.4 Administrador: asignar y supervisar

1. Inicie sesión: verá todos los incidentes. Los de la categoría «Otros» quedan NUEVO hasta que usted los asigne.
2. En el detalle de un incidente elija un técnico de la lista (solo aparecen los que atienden esa categoría) y confirme la asignación o reasignación.
3. Cambie cualquier estado cuando sea necesario.
4. Abra **Dashboard** para ver los totales, los incidentes críticos, los no asignados y la distribución por estado, categoría y técnico.

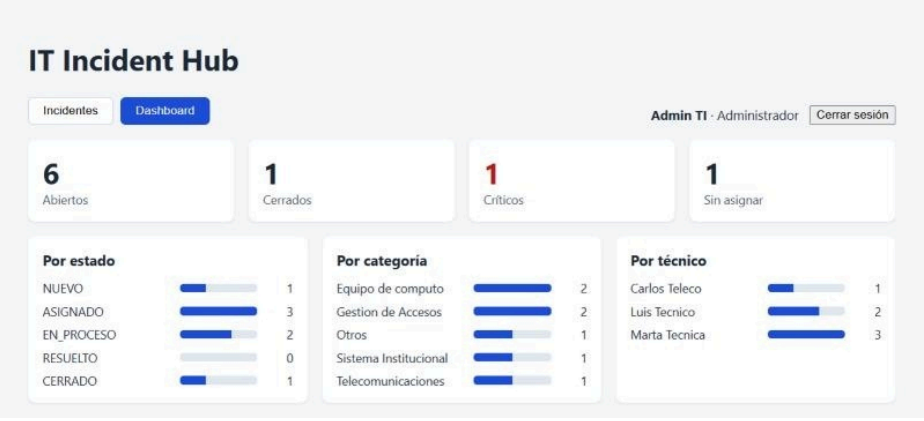


Figura 12. Dashboard del administrador

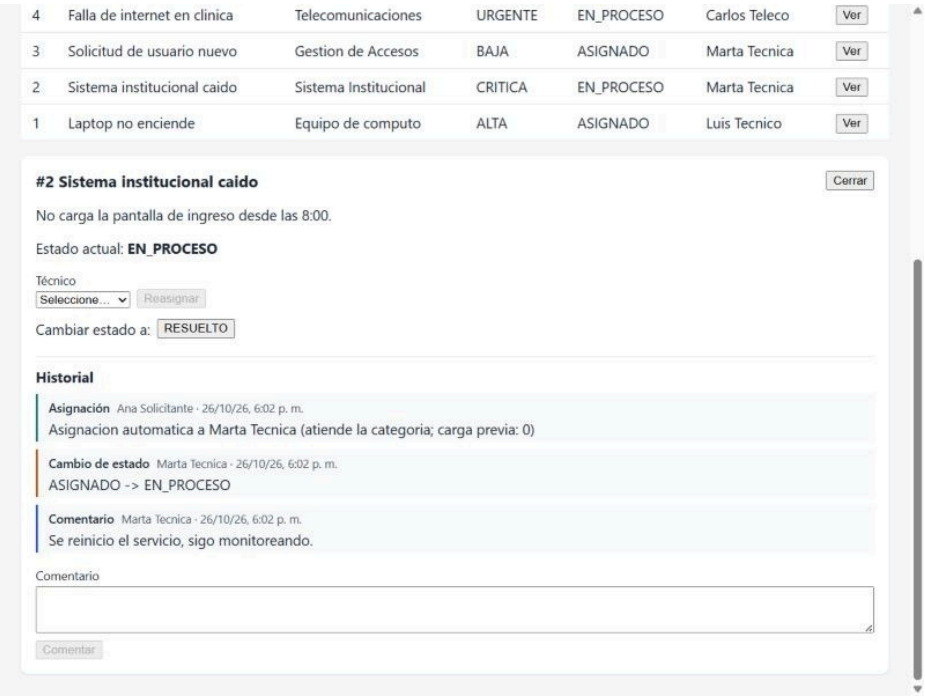


Figura 13. Detalle de un incidente con asignación y bitácora

13.5 Mensajes de la interfaz

Tabla 52. Mensajes frecuentes

Mensaje	Significado y acción
Credenciales inválidas	Correo o contraseña incorrectos; verifique e intente de nuevo
La sesión expiró	Vuelva a iniciar sesión
Transición inválida	El estado elegido no es el siguiente en el flujo; use uno de los permitidos
El incidente debe tener un técnico asignado	Pida a un administrador que asigne un técnico antes de cambiar el estado
El técnico no atiende la categoría	Elija un técnico de la especialidad del incidente

14. Matriz de trazabilidad

La matriz relaciona cada requerimiento con su caso de uso, los componentes que lo implementan y las pruebas automáticas que lo verifican. Permite comprobar que no hay requerimientos sin implementar ni código sin justificación.

Tabla 53. Trazabilidad requerimiento → diseño → prueba

Requerimiento	CU	Componentes	Pruebas
RF-01, RF-02	CU-01	`routes/auth.js`, `auth.js`, `middleware/auth.js`, `Login`, `session.js`, `api.js`	`auth.test`, `auth-routes.test`, `Login.test`, `App.test`, `api.test`
RF-03	CU-02	`routes/incidents.js`, `IncidentForm`	`incidents.test`, `IncidentForm.test`
RF-04	CU-09	`assignment.js`	`auto-assign.test`
RF-05, RF-06	CU-03	`routes/incidents.js`, `access.js`, `IncidentList`, `Filters`	`incidents.test`, `incident-filters.test`, `IncidentList.test`
RF-07	CU-04	`routes/incidents.js`, `IncidentDetail`	`incidents.test`, `IncidentDetail.test`
RF-08	CU-07	`routes/incident-actions.js`, `access.js`, `IncidentDetail`	`incident-actions.test`, `permissions.test`
RF-09	CU-06	`routes/incident-actions.js`, `workflow.js`	`incident-actions.test`, `workflow.test`
RF-10, RF-11	CU-05, CU-04	`routes/incident-history.js`, `IncidentHistory`	`incident-history.test`, `IncidentHistory.test`
RF-12	CU-08	`routes/stats.js`, `Dashboard`	`stats.test`, `Dashboard.test`
RF-13	CU-02, CU-07	`routes/catalogs.js`	`catalogs.test`
RF-14	CU-06	`routes/workflow-route.js`, `workflow.js`	`workflow-route.test`, `workflow.test`
RNF-01, RNF-02	-	`auth.js`, `access.js`, `middleware/auth.js`	`auth.test`, `access.test`, `permissions.test`, `permissions.test` (frontend)
RNF-03	-	`database/migrations`	`schema.test`
RNF-09, RNF-11	-	`check-secrets.js`, pipelines	`tests/secrets.test`, auditoría en CI
RNF-10	-	`deploy.js`, `azure-pipelines.yml`	`tests/deploy.test`

15. Riesgos, deuda técnica y limitaciones

15.1 Registro de deuda técnica

La deuda se registró con Work Items etiquetados `deuda-tecnica` (hijos de la User Story AB#360). La meta del proyecto era identificar al menos cinco y resolver al menos dos con trabajo trazable; se identificaron 11 y se resolvieron 3.

Tabla 54. Deuda técnica

ID	Deuda	Impacto	Estado	Evidencia
D1	`NodeTool@0` deprecado en el pipeline	Medio	**Resuelta**	AB#361, PR #15
D2	Reglas de transición duplicadas en frontend y backend	Medio	**Resuelta**	AB#362, PR #16
D3	Actualización del incidente y evento de bitácora sin transacción	Alto	Abierta	AB#363
D4	Pruebas con pg-mem en vez de PostgreSQL real	Alto	Abierta	AB#364
D5	Sin autenticación: el cliente enviaba `autor_id` y `solicitante_id`	Alto (seguridad)	**Resuelta**	AB#365, v1.1.0
D6	Sin lint en backend ni frontend	Bajo	Abierta	AB#366
D7	`GET /api/incidents` sin paginación	Medio	Abierta	AB#367
D8	El pipeline de Azure no valida los PR automáticamente	Medio	Abierta	AB#368
D9	El filtro por categoría no incluye subcategorías	Bajo	Abierta	AB#369
D10	Sin límite de intentos de login ni bloqueo de cuentas	Medio (seguridad)	Abierta	Pendiente de registrar
D11	Tokens sin revocación y guardados en `sessionStorage`	Medio (seguridad)	Abierta	Pendiente de registrar

Por qué no se resolvieron ahora: D3 y D4 van juntas, porque resolver D3 correctamente exige una transacción y D4 impide probarla con pg-mem; la salida es ejecutar las pruebas contra PostgreSQL real y luego envolver las escrituras en una transacción. D10 y D11 son las siguientes mejoras naturales de seguridad. D6 a D9 son mejoras acotadas para versiones posteriores.

15.2 Riesgos del proyecto

Tabla 55. Riesgos y mitigación

Riesgo	Probabilidad	Impacto	Mitigación
Agente autohospedado no disponible detiene los pipelines	Media	Alto	Guía de registro del agente en `devops/README.md`; las pruebas también corren en GitHub Actions
Aviso de seguridad nuevo rompe el pipeline	Media	Medio	Gate de `npm audit` que obliga a decidir; ya ocurrió y se resolvió (AB#374)

Riesgo	Probabilidad	Impacto	Mitigación
Comportamiento de PostgreSQL no cubierto por las pruebas	Media	Alto	Registrado como D4; plan de pruebas con PostgreSQL real
Exposición de un secreto	Baja	Alto	Escáner en cada corrida; procedimiento de rotación
Fuerza bruta contra el login	Media	Medio	Pendiente (D10); desplegar tras HTTPS y con límite de intentos

15.3 Limitaciones conocidas

- **Revisión de Pull Requests:** la misma identidad aprueba y fusiona; no hay revisor independiente.
- **Equipo simulado:** la distribución de commits por cuenta es desigual y los primeros commits se hicieron con una sola cuenta; el historial no se reescribió.
- **Despliegue local:** los ambientes son carpetas sobre el agente autohospedado, no servidores externos.
- **Docker/PostgreSQL real:** las instrucciones de instalación con Docker no se probaron de punta a punta (Docker estaba apagado durante la verificación); sí está verificado el modo demostración.
- **Aprobaciones de staging y producción:** son manuales; solo se aprobó la corrida final de `main` y las corridas intermedias antiguas permanecen sin aprobar a propósito.
- **Azure Boards:** el acceso de invitados externos está desactivado en la organización universitaria, por lo que los Work Items quedan asignados a la única cuenta de Azure y los roles se indican con etiquetas.

15.4 Trabajo futuro

1. Ejecutar las pruebas contra PostgreSQL real y envolver las escrituras en transacciones (D4, D3).
2. Límite de intentos de login, cookies `HttpOnly` y revocación de tokens (D10, D11).
3. Paginación del listado y búsqueda de texto (D7).
4. Política de rama en Azure Repos con validación automática de Pull Requests (D8) y lint en ambos paquetes (D6).
5. Pantalla de administración de usuarios y especialidades, y notificaciones por correo.
6. Despliegue en un servidor real con HTTPS y el check *Branch control* en los ambientes.

16. Gestión del proyecto

16.1 Cronograma de hitos

Tabla 56. Hitos del proyecto

Fecha	Hito
2026-10-17	Repositorios y backlog creados; v0.1.0 a v0.4.0 (modelo de datos, API, CI, frontend, operación)
2026-10-18	v0.5.0 a v0.8.0: funcionalidad completa, deuda técnica, secretos y ambientes con aprobaciones
2026-10-20	v0.9.0 y v0.9.1: versionado unificado y corrección del despliegue; creación de los tags
2026-10-24	**v1.0.0**: primera versión estable
2026-10-25	**v1.1.0**: inicio de sesión con roles y asignación automática; evidencia y paquete de entrega
2026-10-26	Documento de análisis, diseño y operación (este documento)

16.2 Contribuciones por integrante

Tabla 57. Commits en `main` atribuidos a cada cuenta de GitHub

Cuenta	Commits	Responsabilidad
SotoJ08	32	Backend, DevOps, documentación
Edd210	11	Pruebas, seguridad y permisos
DharaGar29	3	Frontend

Los datos provienen de `docs/evidencia/contribuciones.txt`. La distribución es desigual y se reporta tal cual.

16.3 Evidencias disponibles

Tabla 58. Índice de evidencias

Evidencia	Ubicación
Backlog completo (62 elementos)	`docs/evidencia/backlog-azure-boards.csv`
Pull Requests y sus validaciones	`docs/evidencia/pull-requests.csv`
Corridas de Azure Pipelines	`docs/evidencia/pipeline-runs.csv`
Corridas de GitHub Actions	`docs/evidencia/github-actions-runs.csv`
Historial de Git, ramas y tags	`docs/evidencia/historial-git.csv`, `ramas.txt`, `tags.txt`, y el bundle `incident-hub.git.bundle`
Autoría por cuenta	`docs/evidencia/contribuciones.txt`
Cumplimiento contra el enunciado	`docs/entrega.md`
Diagnóstico de fallos del pipeline	`docs/ci-failure-diagnosis.md`

16.4 Conclusiones

IT Incident Hub cumple el alcance funcional definido y demuestra un ciclo DevOps trazable de punta a punta: cada cambio nace en un Work Item, avanza por rama y Pull Request, pasa un pipeline con pruebas, escáner de secretos y análisis de dependencias, se versiona con SemVer y se promueve por ambientes con aprobaciones. La seguridad se aplica en el servidor y está cubierta por pruebas. Las limitaciones restantes (transacciones y pruebas con PostgreSQL real, protección contra fuerza bruta, despliegue en un servidor externo y revisión independiente de los Pull Requests) están identificadas, justificadas y registradas como deuda técnica.